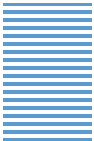




现代密码学

第五章 消息认证和哈希函数

信息与软件工程学院

A decorative blue horizontal bar with a series of parallel lines is positioned to the left of the main title.

主要内容

5.1 消息认证码

5.2 哈希函数

5.3 SHA-1哈希算法

5.4 SHA-3哈希算法

5.5 SM3 哈希算法

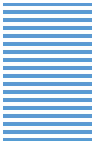
5.6 HMAC算法



现代密码学

消息认证码

信息与软件工程学院

A decorative blue horizontal bar with a series of parallel lines is positioned to the left of the title.

消息认证

- 抗击被动攻击：加密
 - 抗击主动攻击：消息认证
- 消息认证是一个过程，用以验证接收消息的真实性和完整性，同时还用于验证消息的顺序性和时间性。
-

消息认证

- 消息认证机制需要产生**认证符**。
 - **认证符**：用于认证消息的数值。
 - **认证符的产生方法**：**消息认证码MAC** (message authentication code) 和**杂凑函数** (hash function) 两大类。
-

消息认证码

消息认证码的定义及使用方式

- **消息认证码**：指消息被一**密钥控制**的公开函数作用后产生的用作认证符的**固定长度**的数值，或称为密码校验和。

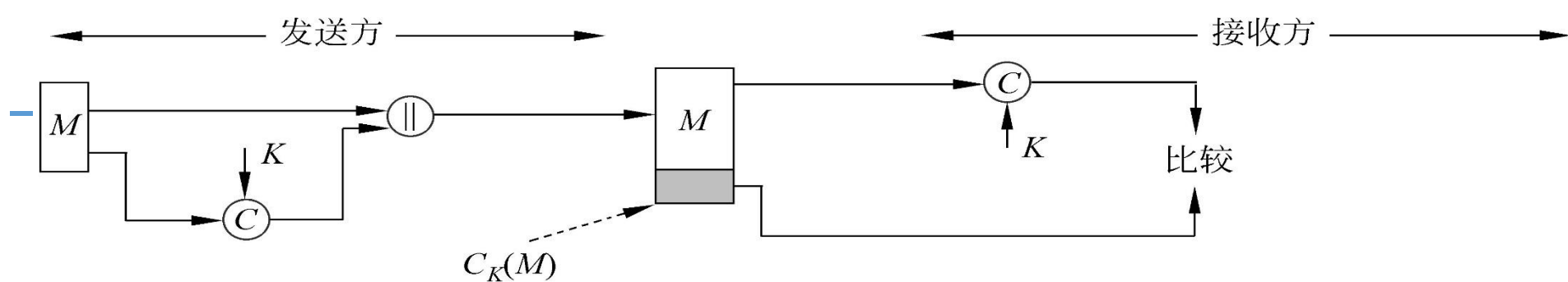
$$M || C_K(M)$$

- 此时需要通信双方A和B共享一密钥K。
- 如果仅收发双方知道K，且B计算得到的MAC与接收到的MAC一致，则这一就实现了以下功能：
 - ① 接收方相信发送方发来的消息未被篡改
 - ② 接收方相信发送方不是冒充的。

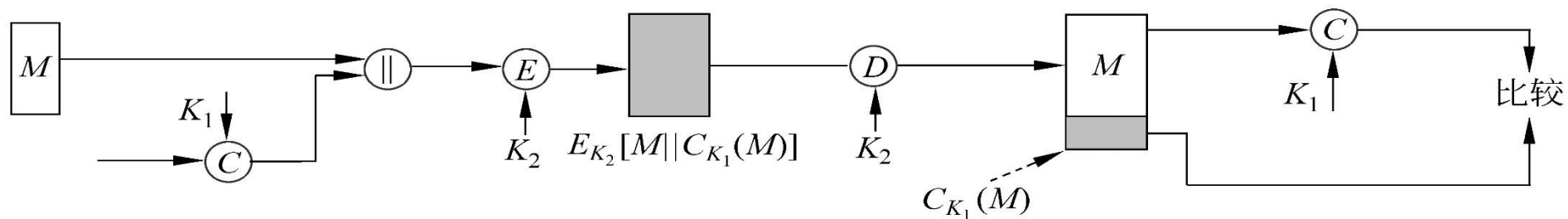
接收方能向第3方证明
消息的来源吗？

消息认证码

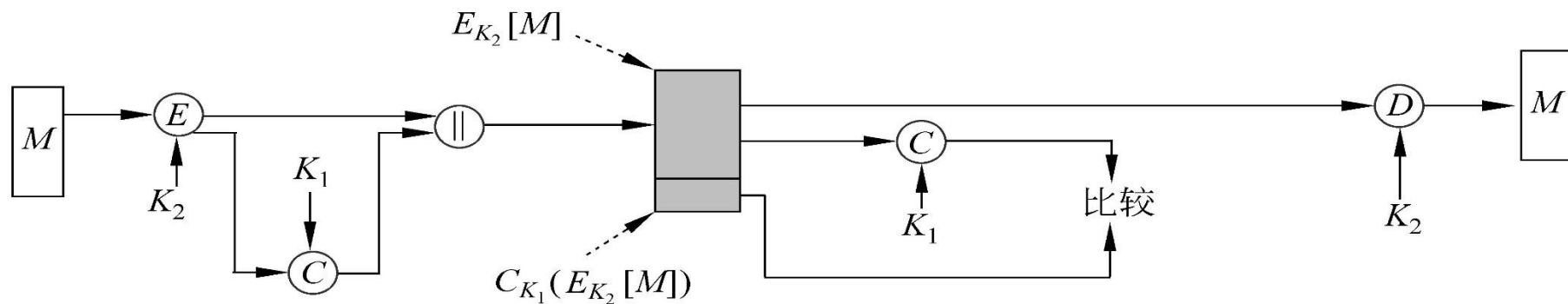
- MAC函数与加密算法类似，不同之处为MAC函数不必是可逆的，因此与加密算法相比更不易被攻破。
 - 上述过程中，由于消息本身在发送过程中是明文形式，所以这一过程只提供认证性而未提供保密性。
-



(a) 消息认证



(b) 认证性和保密性：对明文认证



(c) 认证性和保密性：对密文认证

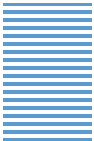
图5.1 MAC的基本使用方式

产生MAC的函数应满足的要求

(1) 消息认证码的穷搜索攻击的代价比使用相同长度密钥的加密算法的穷搜索攻击还要大。

已知 $M \parallel C_K(M) = MAC$, 找 K 。

- 第1轮 已知 M_1 、 MAC_1 ，其中 $MAC_1 = C_K(M_1)$ 。对所有 2^k 个可能的密钥计算 $MAC_i = C_{K_i}(M_1)$ ，得 2^{k-n} 个可能的密钥。
- 第2轮 已知 M_2 、 MAC_2 ，其中 $MAC_2 = C_K(M_2)$ 。对上一轮得到的 2^{k-n} 个可能的密钥计算 $MAC_i = C_{K_i}(M_2)$ ，得 $2^{k-2 \times n}$ 个可能的密钥。
- 如此下去，如果 $k = \alpha n$ ，则上述攻击方式平均需要 α 轮。例如，密钥长为80比特，MAC长为32比特，则第1轮将产生大约 2^{48} 个可能密钥，第2轮将产生 2^{16} 个可能的密钥，第3轮即可找出正确的密钥。

A decorative blue horizontal bar with a series of parallel lines is positioned to the left of the section header.

产生MAC的函数应满足的要求

(2)有些攻击法却不需要寻找产生MAC所使用的密钥。攻击者可假冒发假消息 $M' \| C_K(M')$ 给接收方。

产生MAC的函数应满足的要求

- **例1:** 设 $M = (X_1 \parallel X_2 \parallel \dots \parallel X_m)$ 是由64比特长的分组 X_i ($i=1, \dots, m$) 链接得到的, 其消息认证码由以下方式得到:

$$\Delta(M) = X_1 \oplus X_2 \oplus \dots \oplus X_m$$

$$C_K(M) = E_K[\Delta(M)]$$

- 其中 $\oplus$ 表示异或运算, 加密算法是电码本模式的DES。因此, 密钥长为56比特, MAC长为64比特, 如果敌手得到 $M \parallel C_K(M)$, 那么敌手使用穷搜索攻击寻找K将需做 2^{55} 次加密。

- 然而敌手还可用以下方式攻击系统: 随机选用 $X_1, \dots, X_{m-1}$

$$X_1, \dots, X_{m-1} \rightarrow Y_1, \dots, Y_{m-1}$$

$$Y_m = Y_1 \oplus Y_2 \oplus \dots \oplus Y_{m-1} \oplus \Delta(M)$$

$$X_m \rightarrow Y_m$$

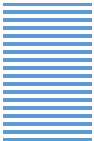
伪造一新消息:

$$M' = Y_1 \parallel Y_2 \parallel \dots \parallel Y_{m-1} \parallel Y_m$$

且 M' 的MAC与原消息 M 的MAC相同。

产生MAC的函数应满足的要求

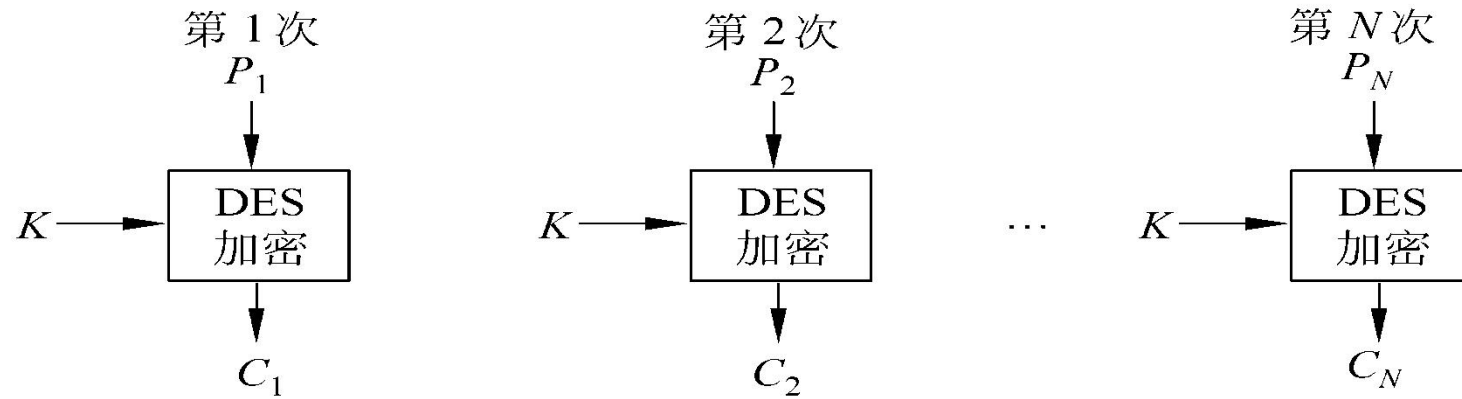
- 考虑到MAC所存在的以上攻击类型，可知它应满足以下要求，其中假定敌手知道函数 C ，但不知道密钥 K ：
 - ① 如果敌手得到 M 和 $C_K(M)$ ，则构造一满足 $C_K(M') = C_K(M)$ 的新消息 M' 在计算上是不可行的。
 - ② $C_K(M)$ 在以下意义下是均匀分布的： 随机选取两个消息 M 、 M' ， $\Pr[C_K(M) = C_K(M')] = 2^{-n}$ ，其中 n 为MAC的长。
 - ③ 若 M' 是 M 的某个变换，即 $M' = f(M)$ ，例如 f 为插入一个或多个比特，那么 $\Pr[C_K(M) = C_K(M')] = 2^{-n}$ 。

A decorative blue horizontal bar with a series of parallel lines is positioned on the left side of the slide.

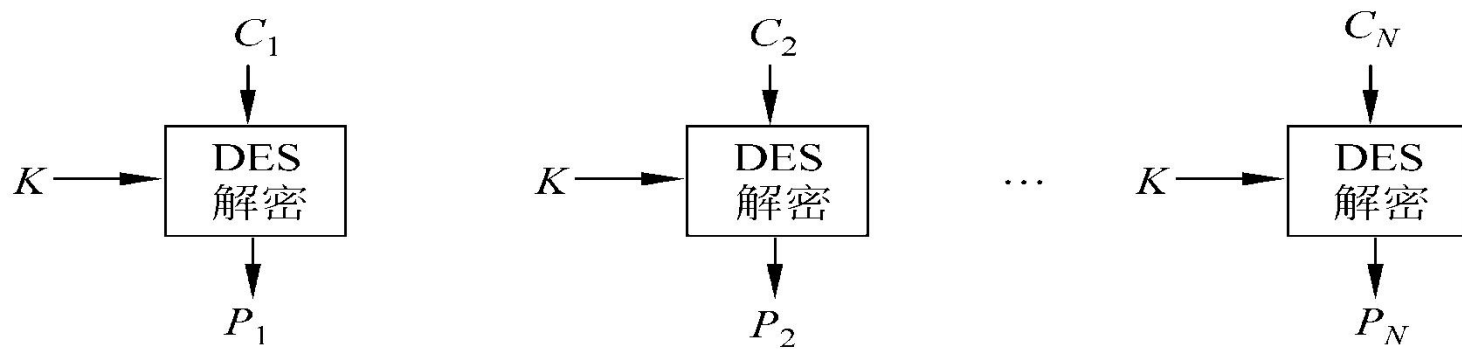
使用DES的工作模式设计MAC

1. 电码本 (ECB) 模式
 2. 密码分组链接 (CBC) 模式
 3. 密码反馈 (CFB) 模式
 4. 输出反馈 (OFB) 模式
 5. 计数器模式
-

电码本ECB (Electronic Code Book) 模式

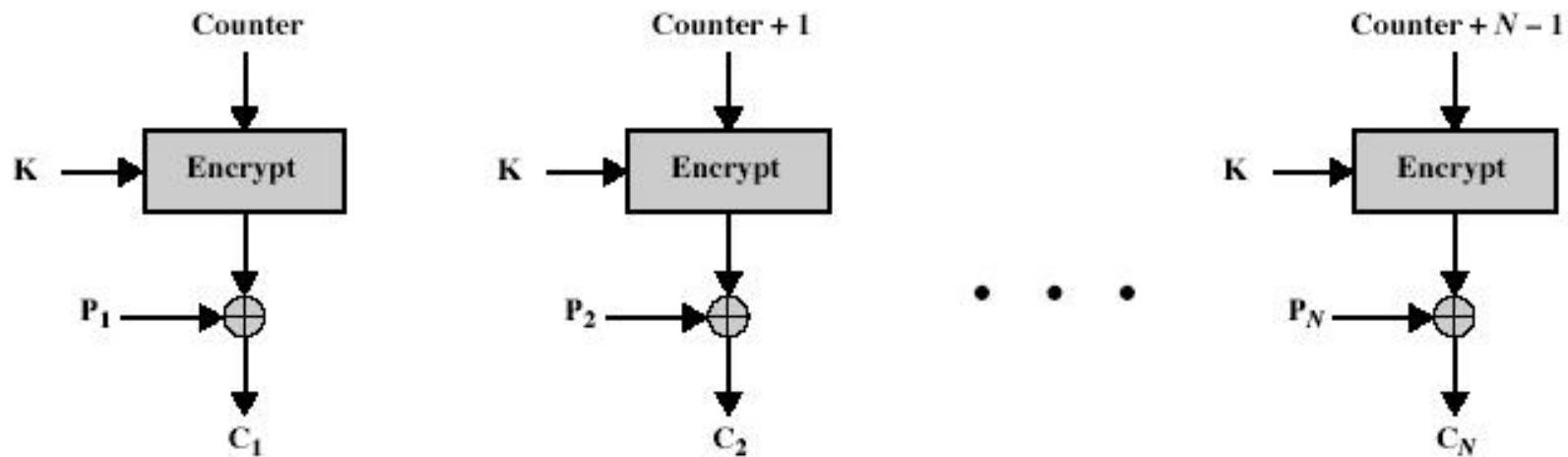


(a) 加密

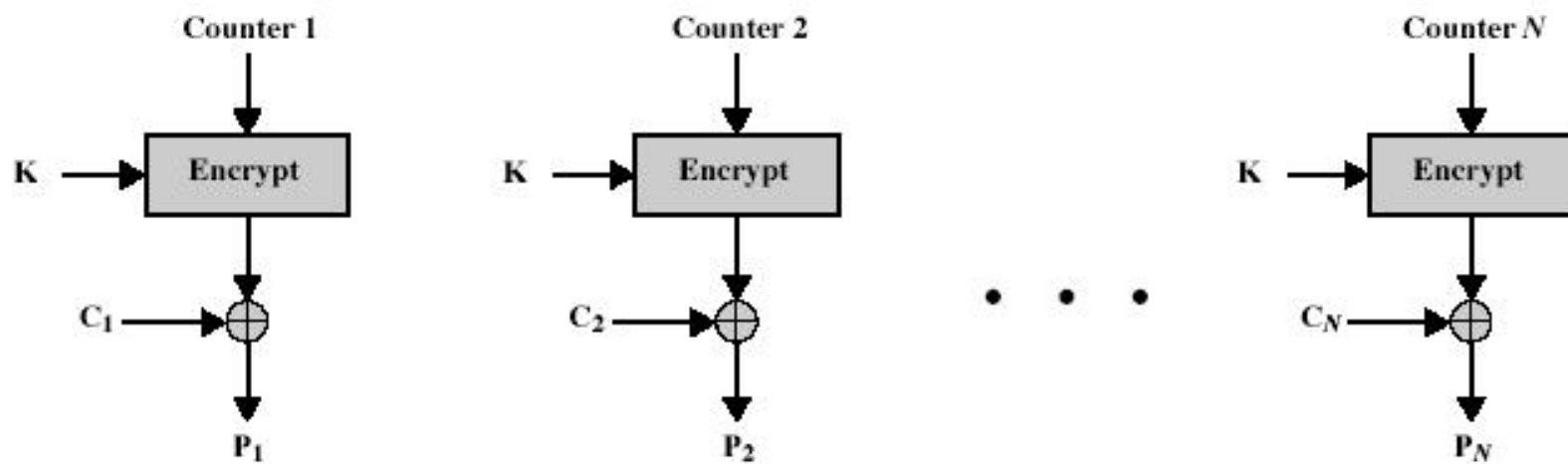


(b) 解密

计数器模式的结构

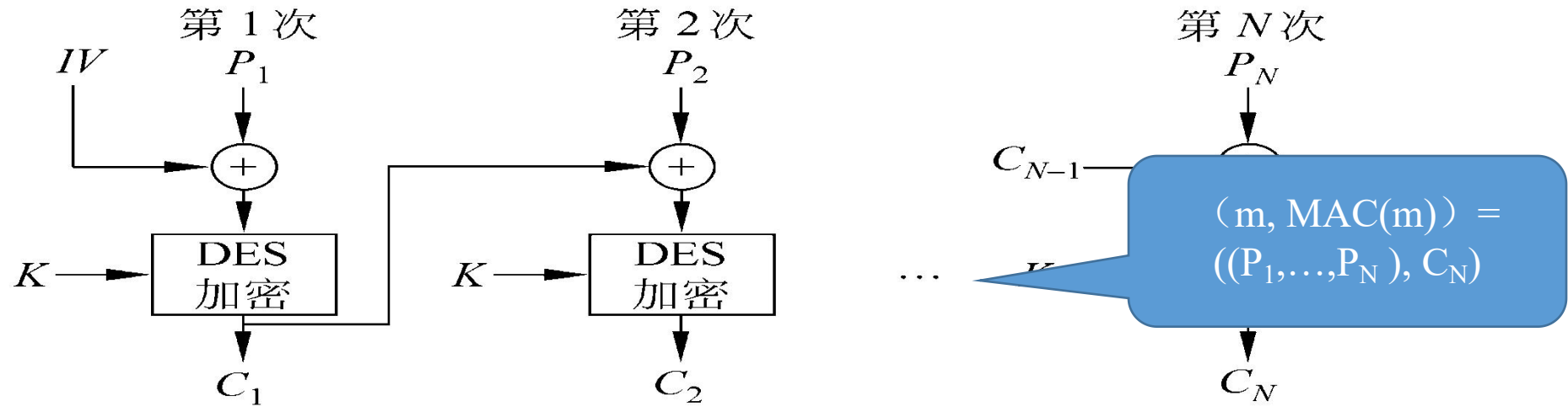


(a) Encryption

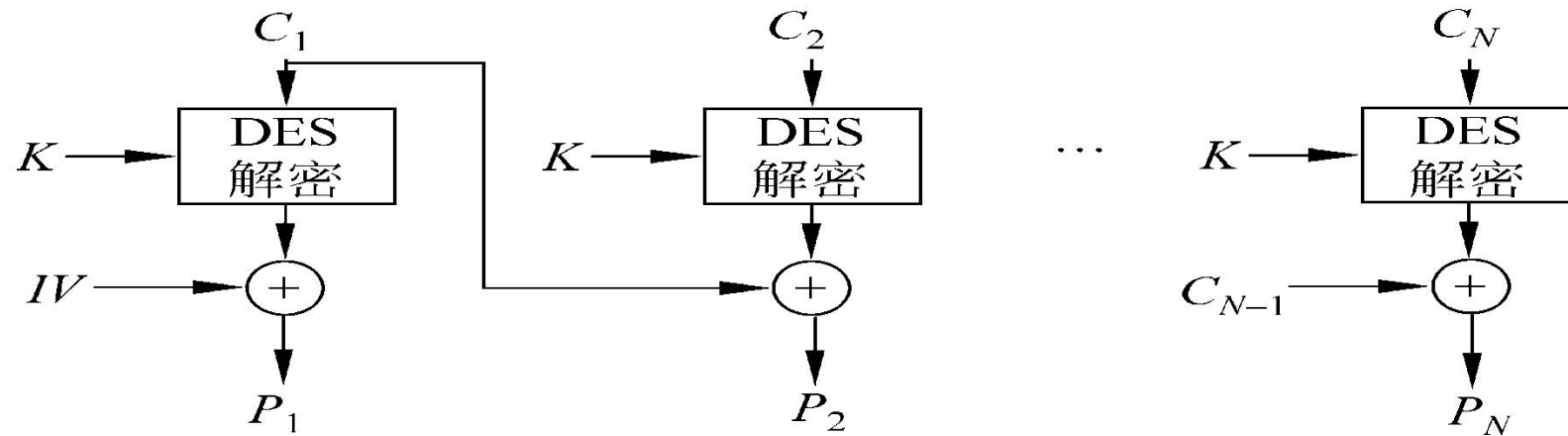


(b) Decryption

密码分组链接CBC (Cipher Block Chaining) 模式



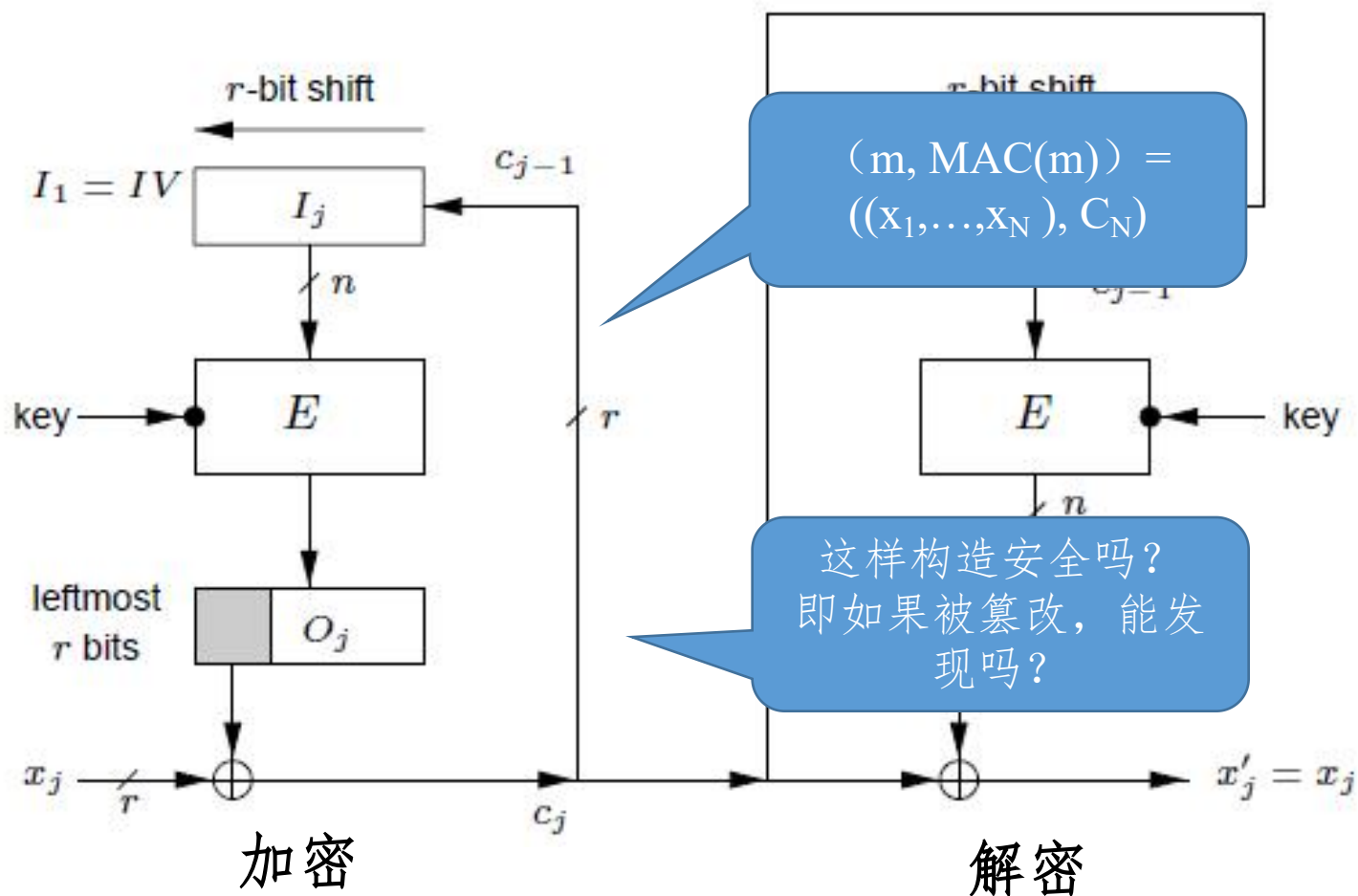
(a) 加密



(b) 解密

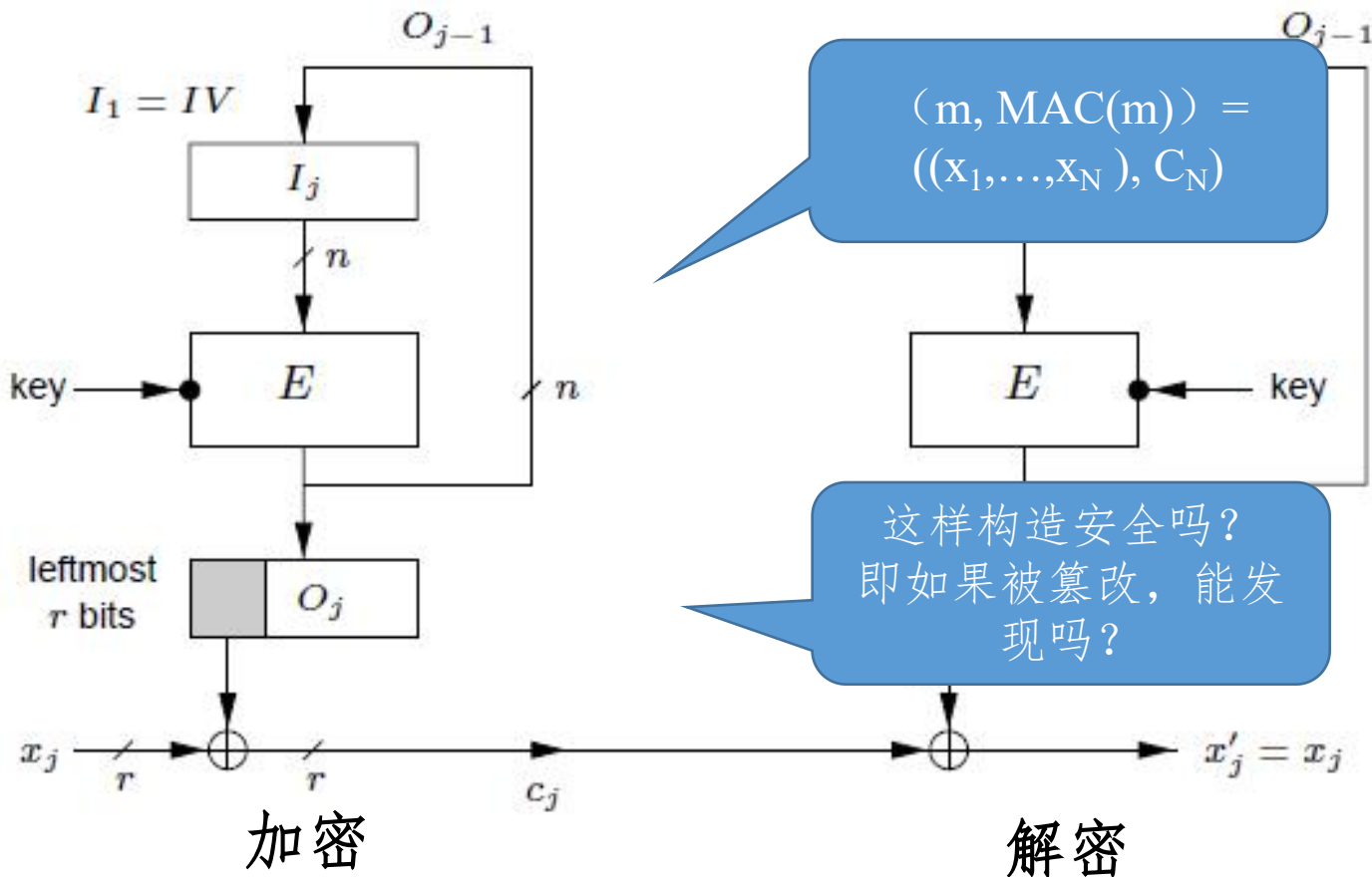
CFB的加密解密

- 若记 $IV=c_{-l+1} \dots c_{-1} c_0$, $|c_i|=r$, 则加密过程可表示为: $c_i = x_i \oplus \text{left}_r(E_k(c_{i-1} \dots c_{i-l}))$



输出反馈OFB (Output Feedback) 模式

- OFB模式在结构上类似于CFB模式，但反馈的内容是DES的输出而不是密文！





现代密码学

哈希函数的基本概念

信息与软件工程学院

Hash函数的定义

- **Hash函数的定义**

- 将任意长的消息 M 映射为较短的、**固定长度**的一个值 $H(M)$ 。
 - **Hash函数**也称为**哈希函数**、**散列函数**、**压缩函数**、**杂凑函数**、**指纹函数**等。其函数值 $H(M)$ 为**哈希值**、**散列值**、**杂凑码**、**指纹**、**消息摘要**等。
 - **Hash函数** H 一般是公开的。
-



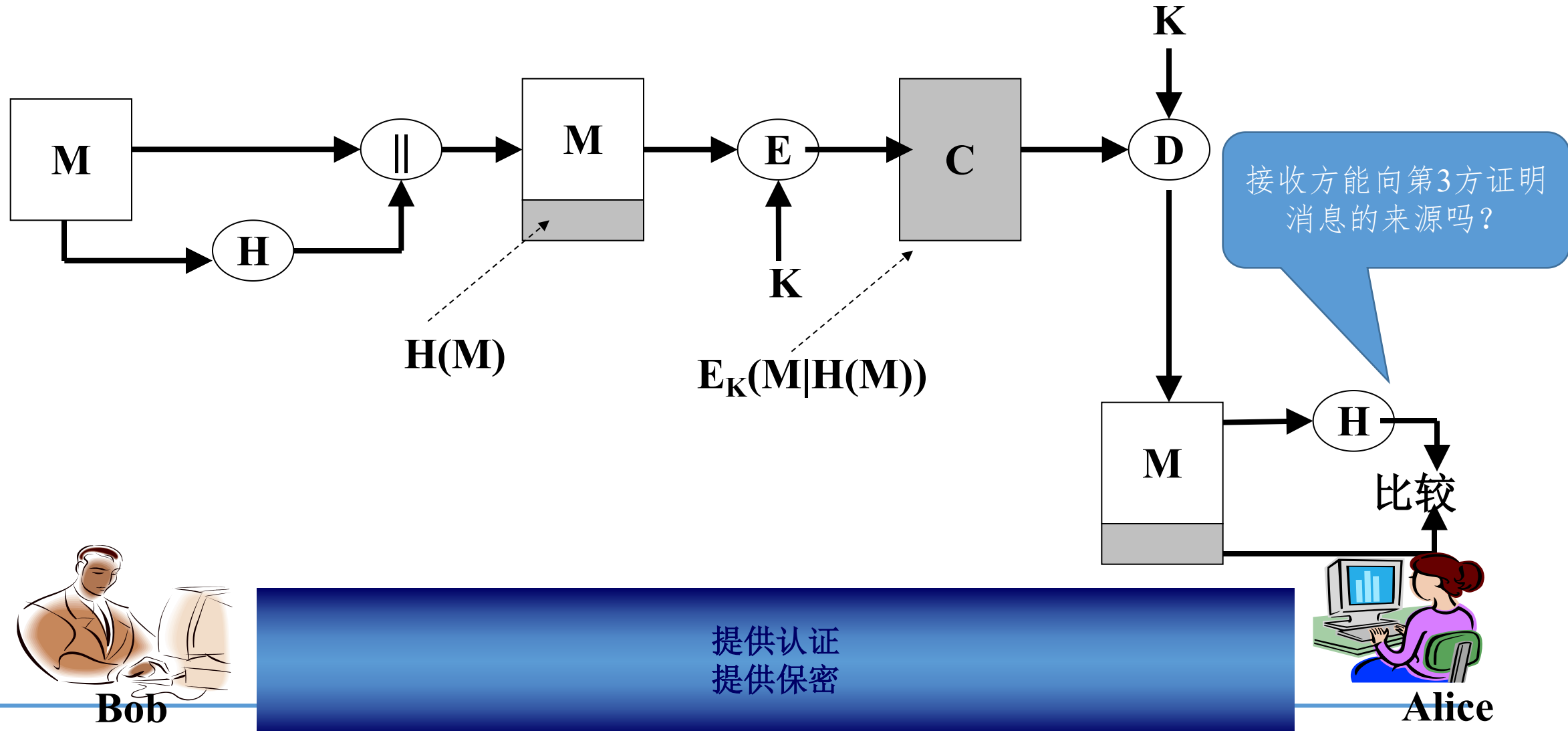
例2



M 是一个长消息，设 $M=(M_1, M_2, \dots, M_k)$ ，其中 M_i 为 l 长的比特串，定义函数 H 如下：

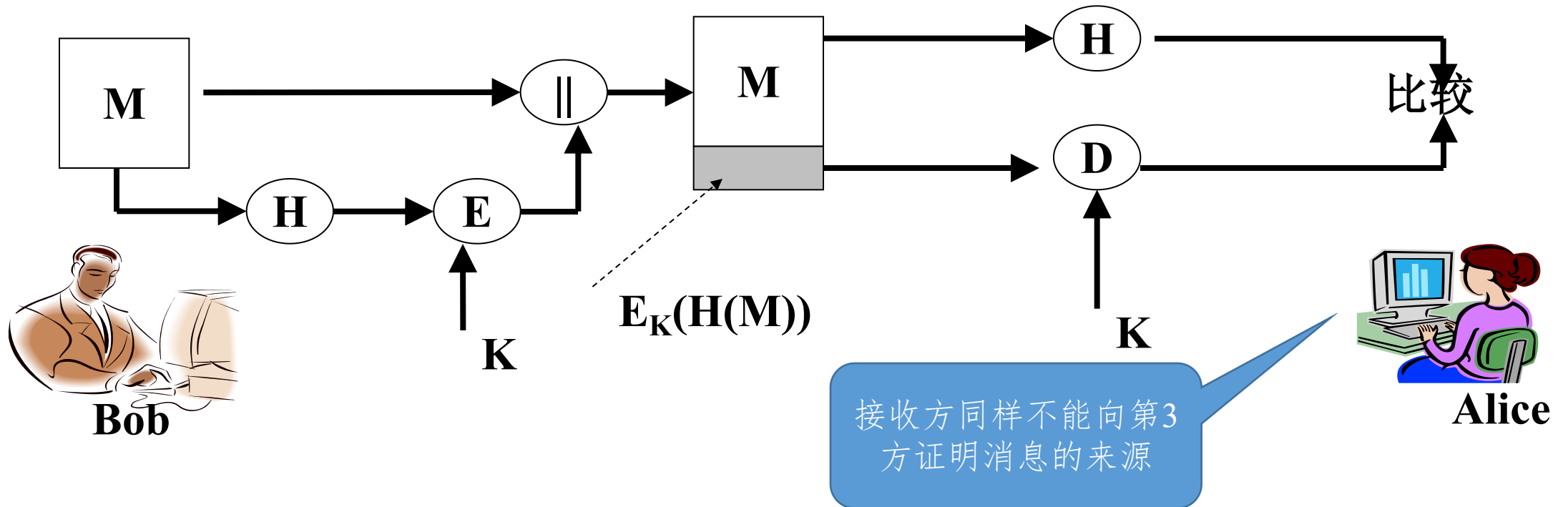
- $$H(M) = M_1 \oplus M_2 \oplus \dots \oplus M_k$$

认证函数：Hash函数的基本用法



认证函数：Hash函数（续）

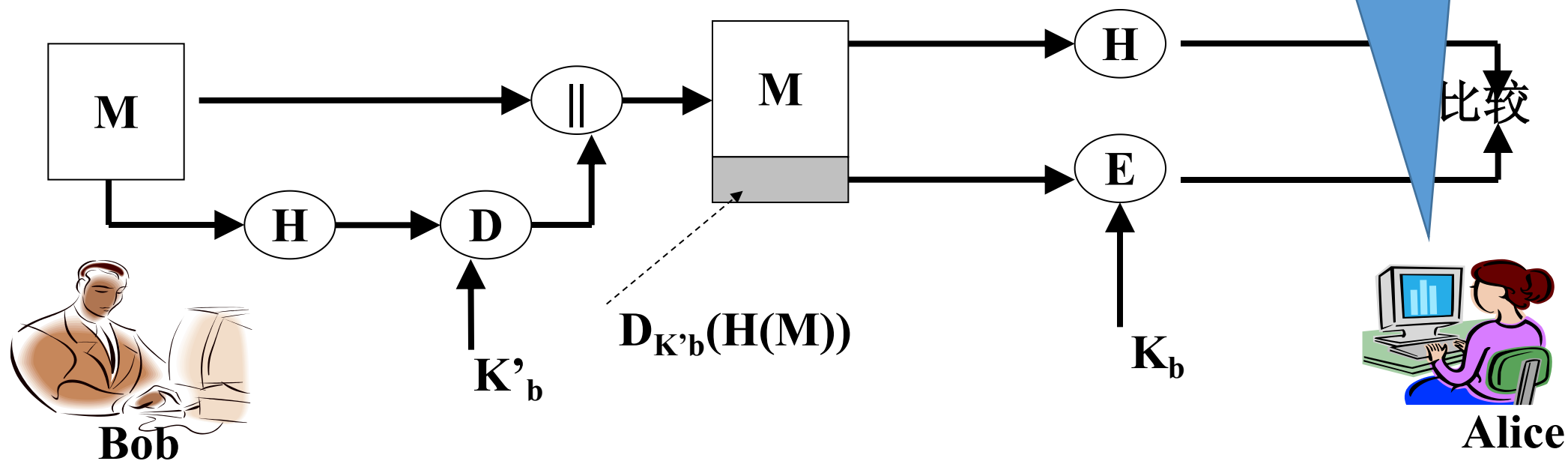
哈希函数的基本用法（b）



提供认证

认证函数：Hash函数（续）

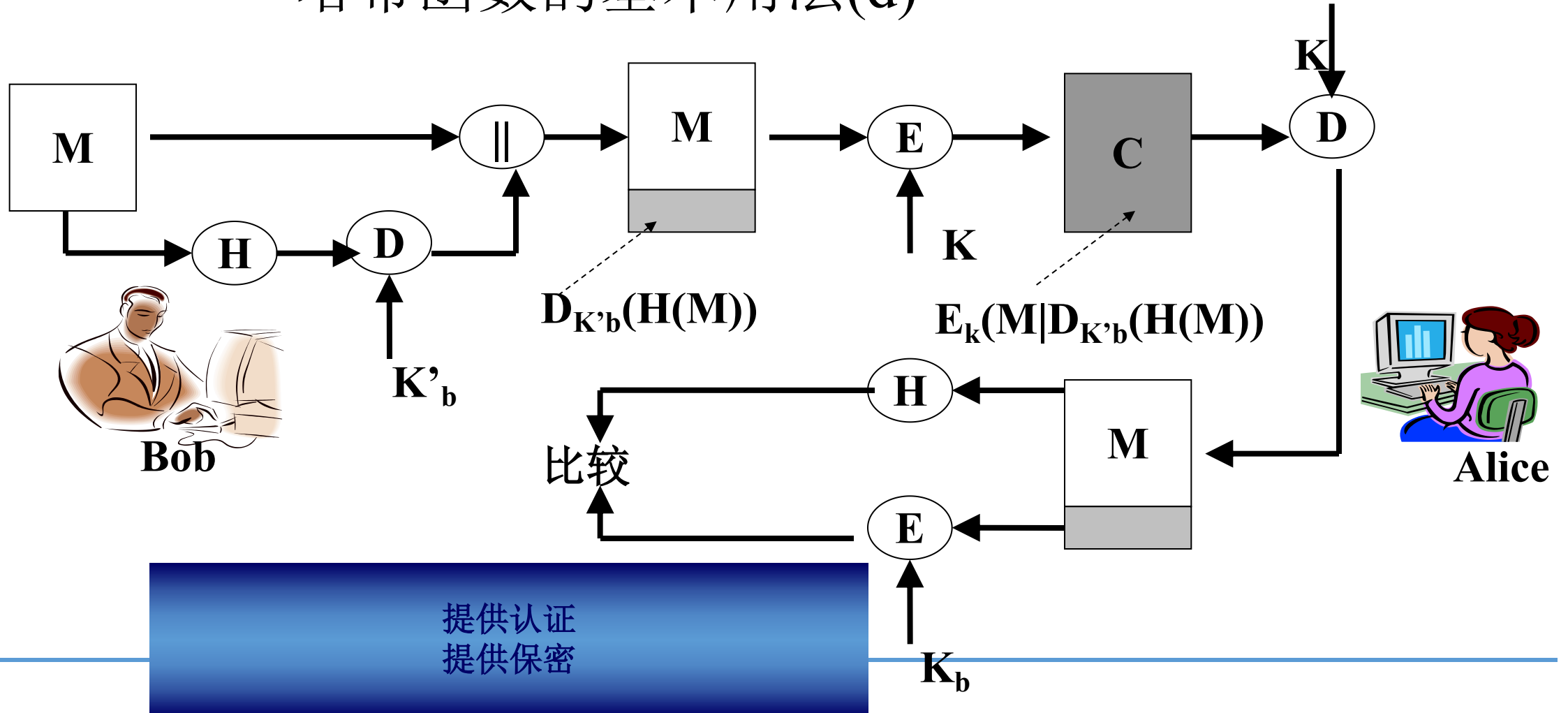
哈希函数的基本用法（c）



提供认证

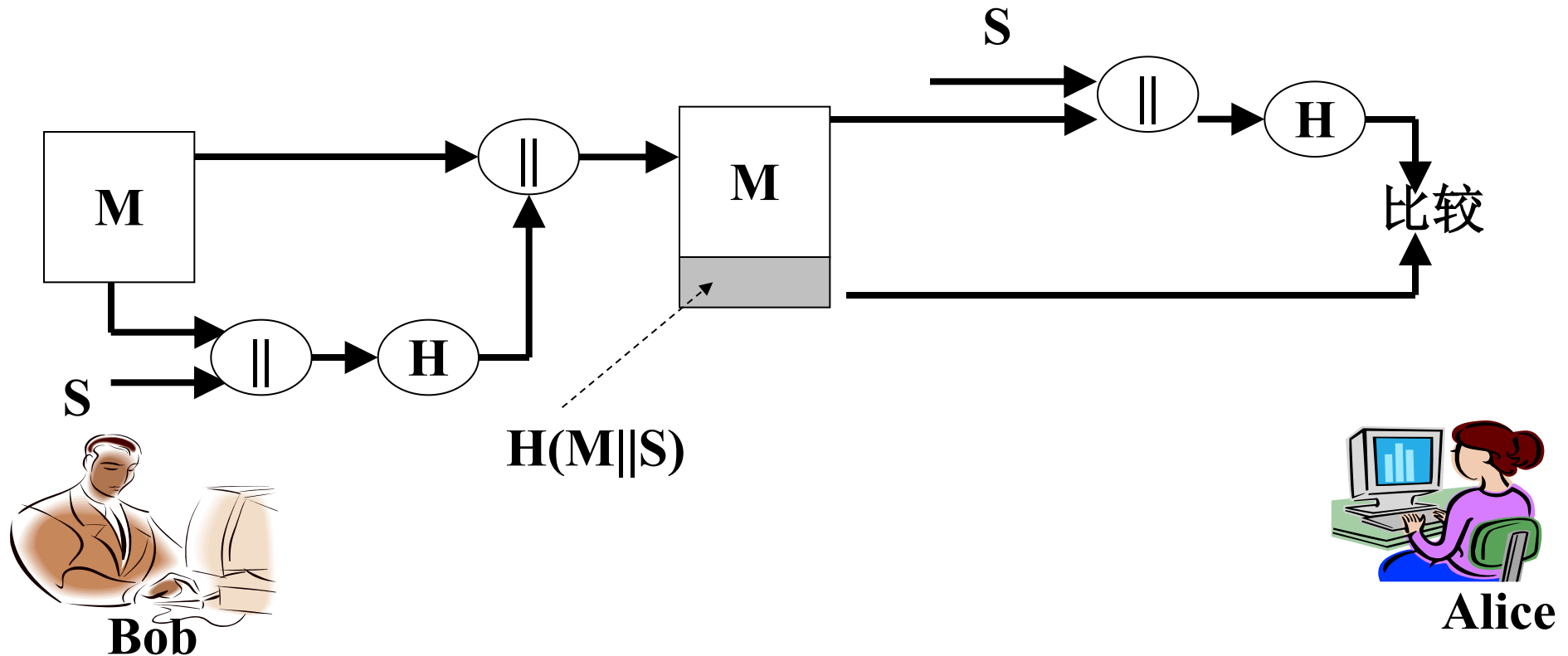
认证函数：Hash函数（续）

哈希函数的基本用法(d)



认证函数：Hash函数（续）

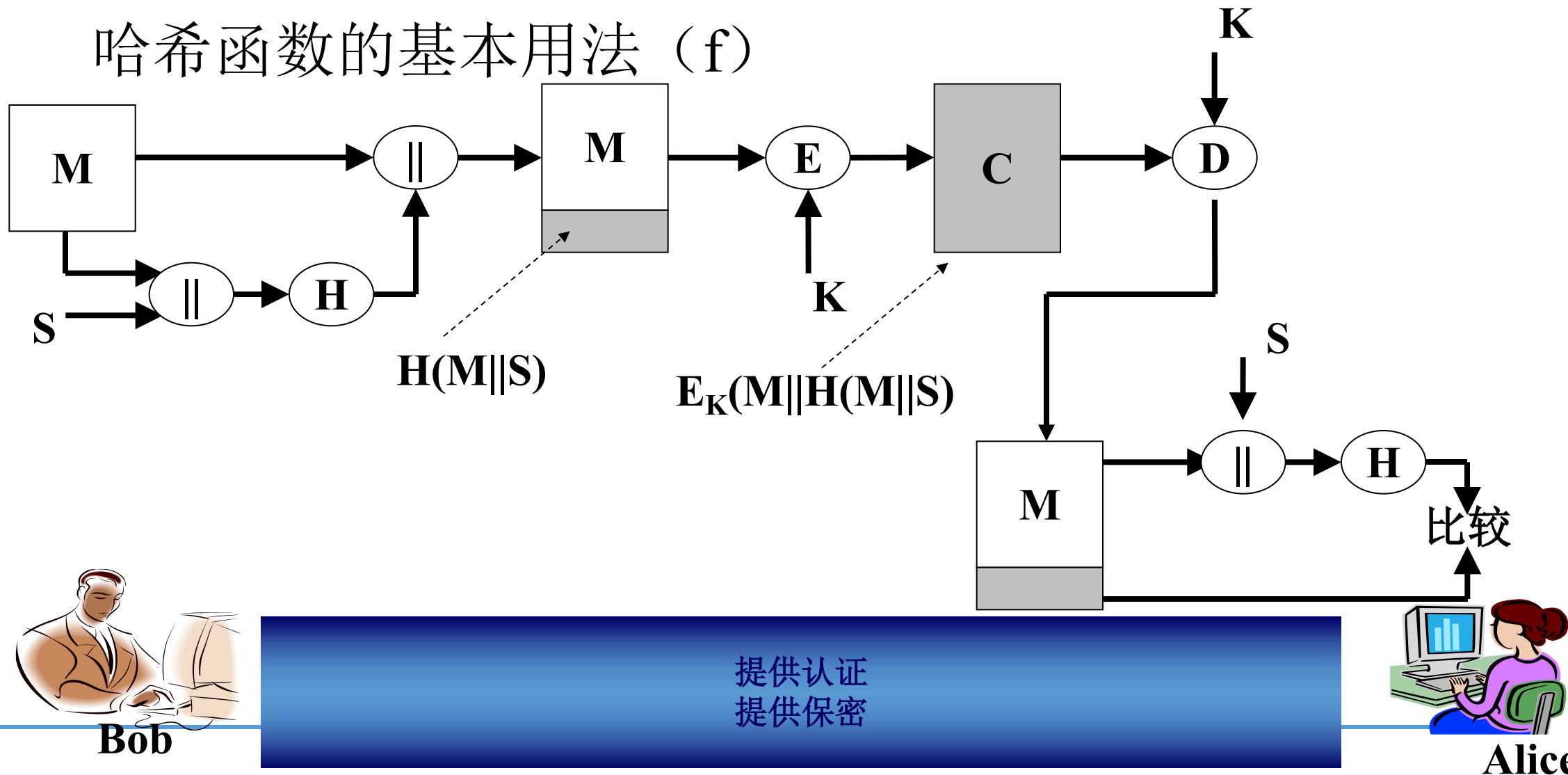
哈希函数的基本用法（e）

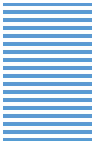


提供认证

认证函数：Hash函数（续）

哈希函数的基本用法 (f)



A decorative blue horizontal bar with a series of parallel lines is positioned on the left side of the slide.

Hash函数满足条件

- **Hash函数的目的是为需认证的数据产生一个“指纹”，Hash函数应满足以下条件：**
 - **Hash函数函数的输入可以是任意长**
 - **Hash函数函数的输出是固定长**
 - **易于在软件和硬件实现**
-



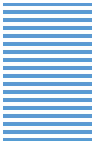
Hash函数满足的安全条件

- 同时，Hash函数为了实现安全认证，需要满足如下安全条件：
 - 单向性：已知 x ，求 $H(x)$ 较为容易；但是，已知 h ，求使得 $H(x)=h$ 的 x 在计算上是不可行的。
 - 抗弱碰撞性：已知 x ，找出 $y(y \neq x)$ 使得 $H(y)=H(x)$ 在计算上是不可行的。
 - 抗强碰撞性：找出任意两个不同的输入 x 、 y ，使得 $H(y)=H(x)$ 在计算上是不可行的。
-



Hash函数满足的安全条件

- 同时，Hash函数为了实现安全认证，需要满足如下安全条件：
 - 单向性：已知 x ，求 $H(x)$ 较为容易；但是，已知 h ，求使得 $H(x)=h$ 的 x 在计算上是不可行的。
 - 单向性问题：已知 $H(x)$ ，求 x
 - 抗弱碰撞性：已知 x ，找出 $y(y \neq x)$ 使得 $H(y)=H(x)$ 在计算上是不可行的。
 - 抗弱碰撞性问题：已知 $H(x)$ ， x ，求 $y (y \neq x)$ 使得 $H(y)=H(x)$
 - 抗强碰撞性：找出任意两个不同的输入 x 、 y ，使得 $H(y)=H(x)$ 在计算上是不可行的。
 - 抗强碰撞性问题：已知 $H(x)$ ，求 x 、 y ，使得 $H(y)=H(x)$
-

A decorative blue horizontal bar with a series of parallel lines is positioned on the left side of the slide.

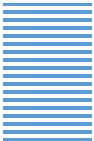
例2（续）

M 是一个长消息，设 $M=(M_1, M_2, \dots, M_k)$ ，其中 M_i 为 l 长的比特串，定义函数 H 如下：

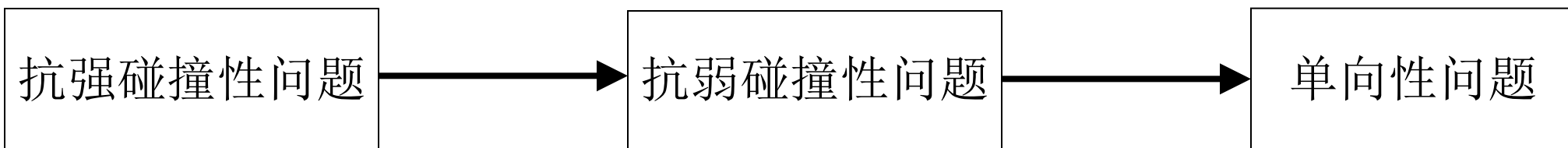
- $$H(M) = M_1 \oplus M_2 \oplus \dots \oplus M_k$$

单选题

- 在Hash函数中，已知 x ，找到 $y(y \neq x)$ 满足 $h(y)=h(x)$ 在计算上是不可行的，这一性质称为_____。
A、完整性 B、单向性 C、抗弱碰撞性 D、抗强碰撞性
- 在Hash函数中，找到任意两个不同输入 $y \neq x$ ，满足 $h(y)=h(x)$ 在计算上是不可行的，这一性质称为_____。
A、完整性 B、单向性 C、抗弱碰撞性 D、抗强碰撞性
- 已知 x ，求 $H(x)$ 较为容易；但是，已知 h ，求使得 $H(x)=h$ 的 x 在计算上是不可行的，这一性质称为_____。
A、完整性 B、单向性 C、抗弱碰撞性 D、抗强碰撞性

A decorative graphic consisting of several horizontal blue bars of varying lengths, stacked vertically, is positioned to the left of the title.

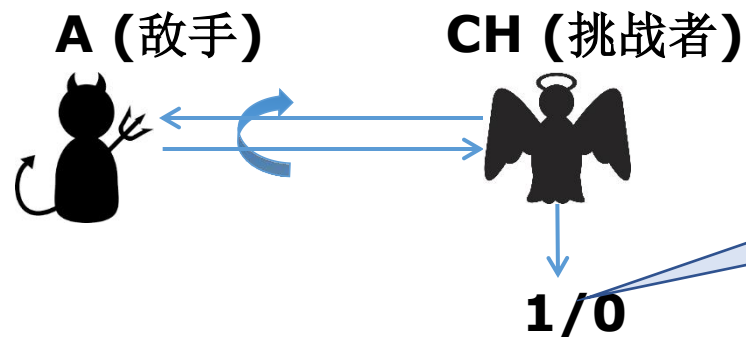
3个安全性的关系



密码游戏与归约

假设

G_1 的可证明安全性：基于 G_2 证明 G_1 的安全性



敌手能攻破密码游戏 $\Leftrightarrow$ 挑战者输出 1

几乎所有的计算性的假设和安全性都能
用密码游戏来表示 $\Rightarrow G_2$ 不安全

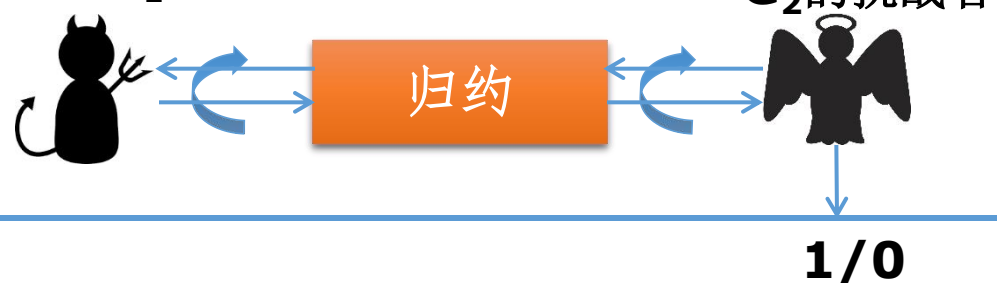
$\Pr[\text{CH 输出 } 1] = \text{negl} \Rightarrow$ 密码游戏是安全的

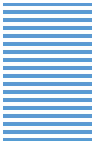
可忽略的概率

存在敌手攻破 $G_1 \Rightarrow$ 存在敌手攻破 G_2

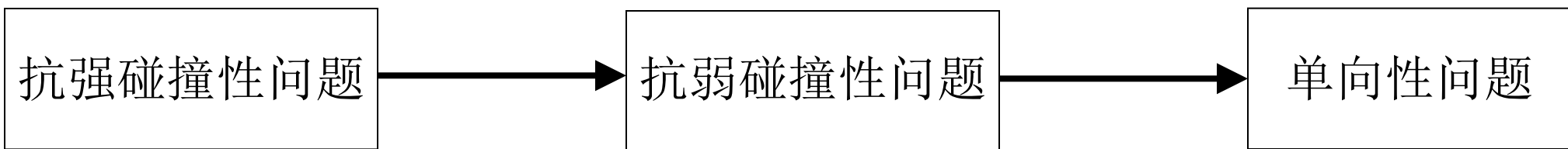
G_1 到 G_2 的归约

攻破 G_1 的敌手

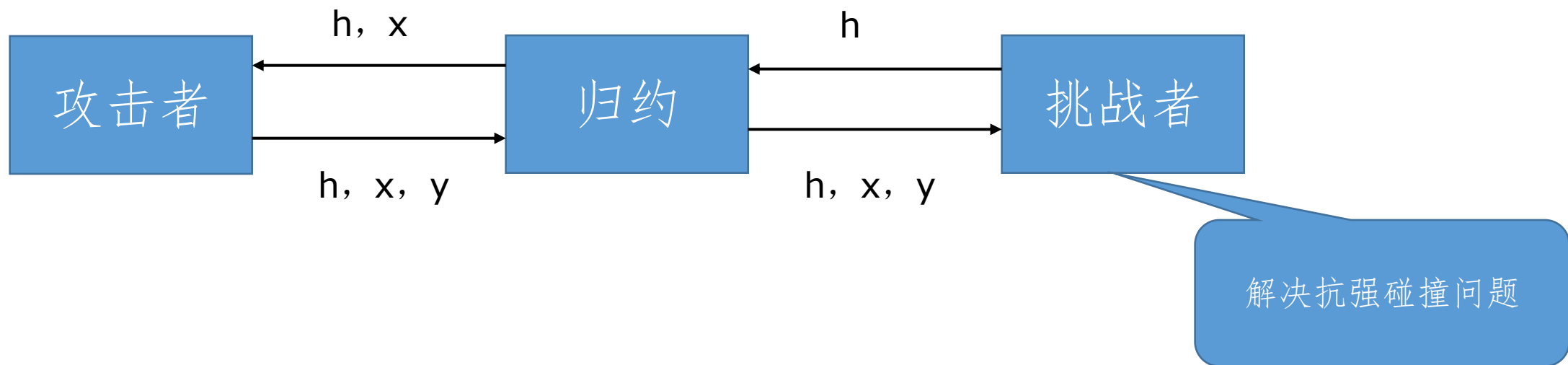


A decorative blue horizontal bar with a series of vertical lines of varying heights, resembling a barcode or a stylized 'E', is positioned to the left of the title.

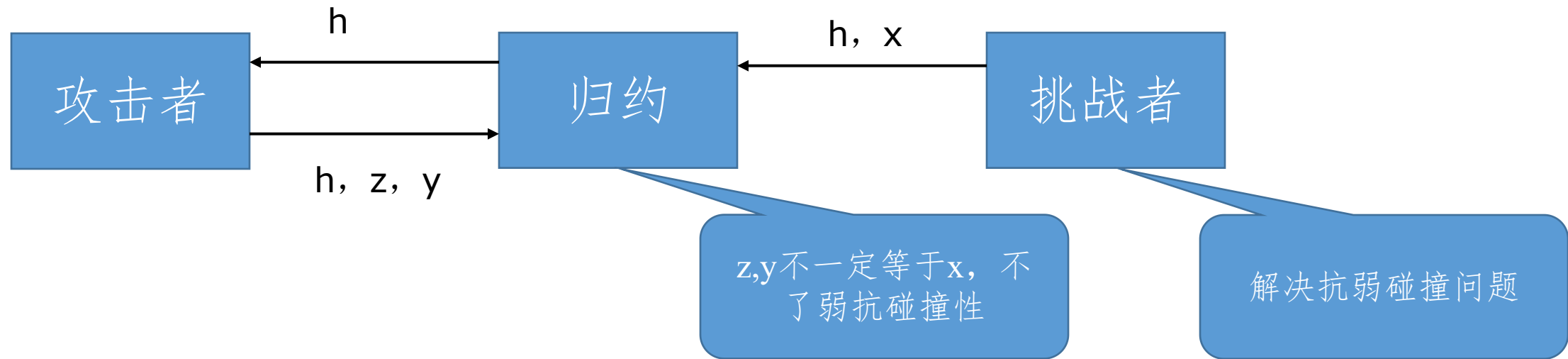
3个安全性的关系



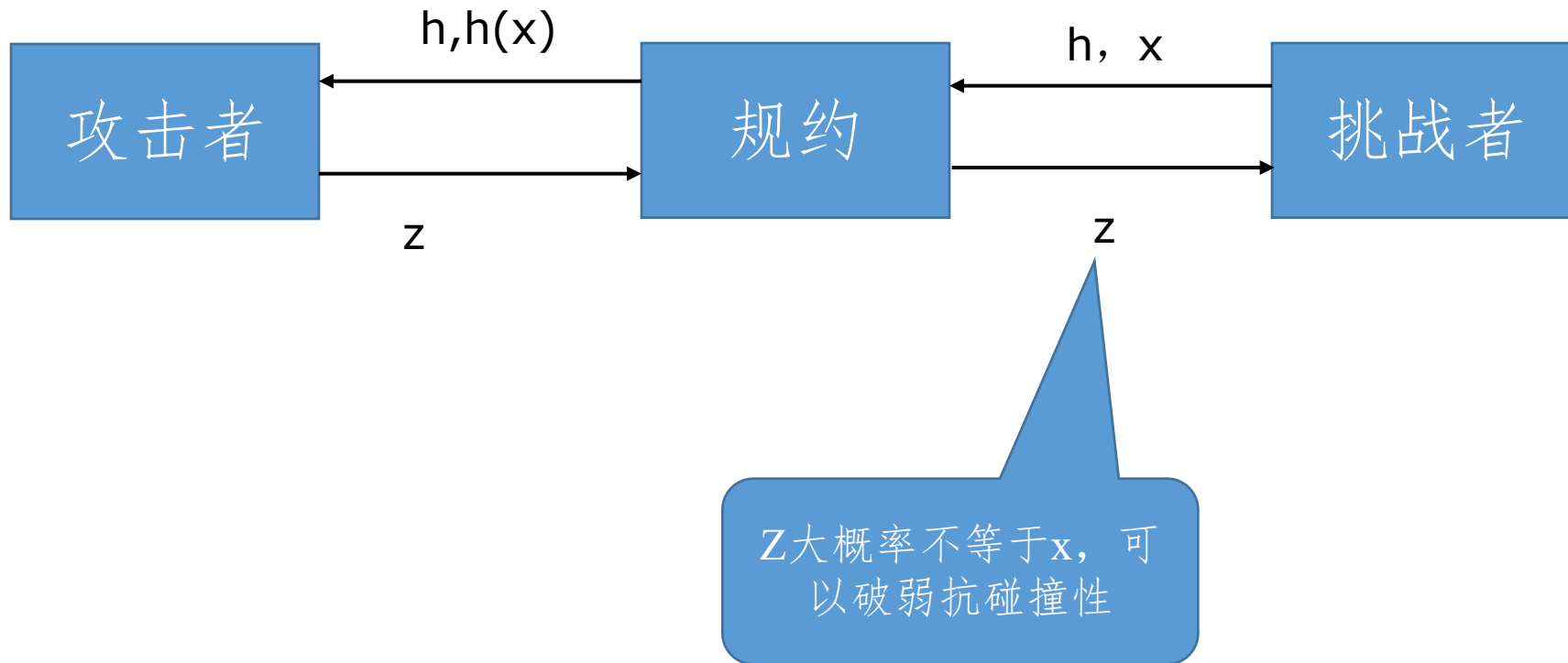
强抗碰撞性到弱抗碰撞性



弱抗碰撞性到强抗碰撞性？



弱抗碰撞性到单向性



Hash函数满足的安全条件

- 同时，Hash函数为了实现安全认证，需要满足如下安全条件：
 - 单向性：已知 x ，求 $H(x)$ 较为容易；但是，已知 h ，求使得 $H(x)=h$ 的 x 在计算上是不可行的。
 - 抗弱碰撞性：已知 x ，找出 $y(y \neq x)$ 使得 $H(y)=H(x)$ 在计算上是不可行的。
 - 抗强碰撞性：找出任意两个不同的输入 x 、 y ，使得 $H(y)=H(x)$ 在计算上是不可行的。
-

生日攻击-相关问题

- 问题1---第 I 类生日攻击问题

- 已知一杂凑函数 H 有 n 个可能的输出， $H(x)$ 是一个特定的输出，如果对 H 随机取 k 个输入，则至少有一个输入 y 使得 $H(y)=H(x)$ 的概率为0.5时， k 有多大？
- 以后为叙述方便，称对杂凑函数 H 寻找上述 y 的攻击为第I类生日攻击。

第一类生日攻击

- 因为 H 有 n 个可能的输出，所以输入 y 产生的输出 $H(y)$ 等于特定输出 $H(x)$ 的概率是 $1/n$ ，反过来说 $H(y) \neq H(x)$ 的概率是 $1-1/n$ 。
- y 取 k 个随机值而函数的 k 个输出中没有一个等于 $H(x)$ ，其概率等于每个输出都不等于 $H(x)$ 的概率之积，为 $[1-1/n]^k$ ，所以 y 取 k 个随机值得到函数的 k 个输出中至少有一个等于 $H(x)$ 的概率为 $1-[1-1/n]^k$ 。

- 由当 $|x| \ll 1$ ， $(1+x)^k \approx 1+kx$ ，可得

$$1-[1-1/n]^k \approx 1-[1-k/n]=k/n$$

- 若使上述概率等于0.5，则 $k=n/2$ 。特别地，如果 H 的输出为 m 比特长，即可能的输出个数 $n=2^m$ ，则

$$k=2^{m-1}$$

生日攻击-相关问题

- 问题2---生日悖论
 - 在 k 个人中至少有两个人的生日相同的概率大于0.5时, k 至少多大?

生日攻击-相关问题

- 问题2---生日悖论
 - 设有 k 个整数项，每一项都在1到 n 之间等可能地取值。 $P(n, k)$: k 个整数项中至少有两个取值相同的概率
 - 生日悖论就是求使得 $P(365, k) \geq 0.5$ 的最小 k 。
 $Q(365, k)$: k 个数据项中任意两个取值都不同的概率。

生日攻击-相关问题

• 问题2---生日悖论

- 如果 $k > 365$ ，则不可能使得任意两个数据都不相同，因此假定 $k \leq 365$ 。 k 个数据项中任意两个都不相同的所有取值方式数为

$$365 \times 364 \times \cdots \times (365 - k + 1) = \frac{365!}{(365 - k)!}$$

- 如果去掉任意两个都不相同这一限制条件，可得 k 个数据项中所有取值方式数为 365^k 。所以可得

$$Q(365, k) = \frac{365!}{(365 - k)! 365^k}$$

$$P(365, k) = 1 - Q(365, k) = 1 - \frac{365!}{(365 - k)! 365^k}$$

生日攻击-相关问题

- 问题2---生日悖论

- 当 $k=23$ 时, $P(365,23)=0.5073$, 即上述问题只需23人, 人数如此之少。
- 若 $k=100$, 则 $P(365,100)=0.9999997$, 即获得如此大的概率。
- 之所以称这-问题是悖论是因为当人数 k 给定时, 得到的至少有两个人的生日相同的概率比想象的要大得多。

生日攻击-相关问题

- 问题---生日悖论推广问题

- 已知一个在1到 n 之间均匀分布的整数型随机变量，若该变量的 k 个取值中至少有两个取值相同的概率大于0.5，则 k 至少多大？

$$P(n, k) = 1 - \frac{n!}{(n-k)!n^k}$$

- 与上类似， $k = 1.18\sqrt{n} \approx \sqrt{n}$ ，令 $P(n, k) > 0.5$ ，可得
- 若取 $n=365$ ，则 $k = 1.18\sqrt{365} = 22.54$ 。

生日攻击

- 生日攻击

- 设杂凑函数H有 2^m 个可能的输出（即输出长 m 比特），如果H的 k 个随机输入中至少有两个产生相同输出的概率大于0.5，则

$$k \approx \sqrt{2^m} = 2^{m/2}$$

- 第II类生日攻击：寻找函数H的具有相同输出的两个任意输入的攻击方式。

第 II 类生日攻击

$A \longrightarrow B$

敌手对 M 产生出 $2^{m/2}$ 个变形消息: $\tilde{M}$

敌手还准备一个假冒的消息 M' ,
产生出 $2^{m/2}$ 个变形的消息:

将 提交给 A 请求签名 $E_{sk_A}(H(\tilde{M}))$

$A \longrightarrow B$

安全应用

- 输出长度与碰撞
 - 这种生日攻击给出了消息摘要长度的下界，一个**40**比特的消息摘要非常不安全的。
 - 因为在大约 2^{20} 个（大约**100万**）个随机值中就能以**1/2**的概率找到一个碰撞。
 - 通常建议消息摘要的最小可接受的长度为**160**比特，在**DSS**签名标准中使用**160**比特的消息摘要就是基于这个考虑。



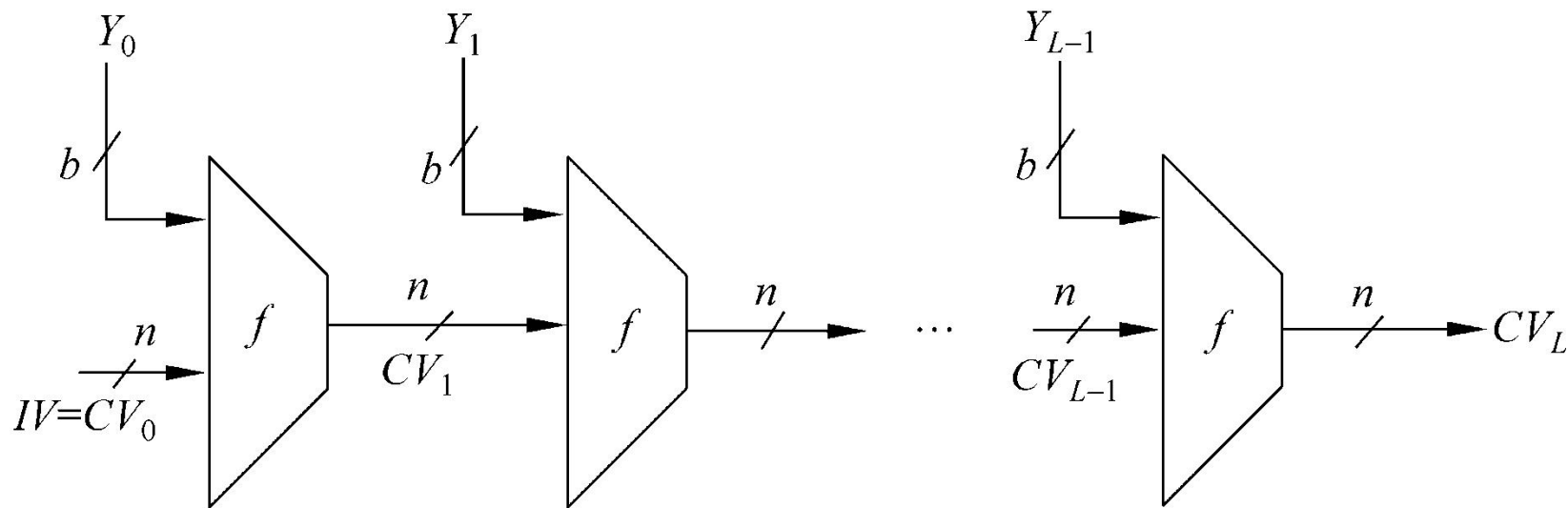
现代密码学

SHA-1 密码杂凑函数

信息与软件工程学院

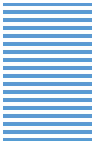
迭代型哈希函数的一般结构

• 迭代



$CV_0 = IV = n$ 比特长的初值;
 $CV_i = f(CV_{i-1}, Y_{i-1}) 1 \leq i \leq L$;
 $H(M) = CV_L$

CV_{i-1} , 称为**链接变量**
 通常 $b > n$, 称函数 f 为**压缩函数**
 分析时需要先**找出 f 的碰撞**

A decorative blue horizontal bar with a striped pattern is positioned to the left of the title.

MD5哈希算法

- **MD4是MD5哈希算法的前身**，由**Ron Rivest**于**1990年10月**作为**RFC**提出，**1992年4月**公布的**MD4**的改进（**RFC 1320, 1321**）称为**MD5**。
 - **输入**：任意长的消息（图中为K比特）
 - **分组**：512比特
 - **输出**：128比特的消息摘要。
-

**CRYPTO'04 会议中,我国学者王小云
在Aug.17 (19:00pm—24:00pm)**

Rump Session Program

**给出15分钟的演讲,
对 MD5 给出了有效地攻击!**

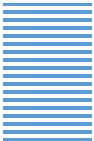


MD5的安全性

- Rivest猜想作为128比特长的哈希值来说，MD5的强度达到了最大，比如说找出具有相同哈希值的两个消息需执行 $O(2^{64})$ 次运算，而寻找具有给定哈希值的一个消息需要执行 $O(2^{128})$ 次运算。
- 然而，2004年，山东大学王小云等成功找出了MD5的碰撞，发生碰撞的消息是由两个1024比特长的串 M 、 N_i 构成，设消息 $M\|N_i$ 的碰撞是 $M'\|N'_i$ ，在IBM P690上找 M 和 M' 花费时间大约一小时，找出 M 和 M' 后，则只需15秒至5分钟就可找出 N_i 和 N'_i 。

安全杂凑算法SHA-1

- 安全哈希算法 (Secure Hash Algorithm, SHA) 由美国NIST设计, 于1993年作为联邦信息处理标准公布。SHA是基于MD4的算法, 其结构与MD4非常类似。
- SHA是基于MD4的算法, 其结构与MD4非常类似
 - SHA在发布之后很快就被 NSA 撤回, 并且以 1995年发布的修订版本 FIPS PUB 180-1 (通常称为 “SHA-1”) 取代
 - 根据 NSA 的说法, 它修正了一个在原始算法中会降低密码安全性的错误。然而 NSA 并没有提供任何进一步的解释或证明该错误已被修正
 - 1998年, 在一次对 SHA-0 的攻击中发现这次攻击并不能适用于 SHA-1 — 我们不知道这是否就是 NSA 所发现的错误, 但这或许暗示我们这次修正已经提升了安全性。
 - SHA-1 已经被公众密码社群做了非常严密的检验而还没发现到有不安全的地方,
 - 但现在理论上已经被王小云破译了

A decorative blue horizontal bar with white horizontal stripes is positioned to the left of the title.

算法描述

- 算法的输入：小于 2^{64} 比特长的任意消息，分为512比特长的分组。
- 算法的输出：160比特长的消息摘要。
- 算法的框图与MD5一样，但杂凑值的长度和链接变量的长度为160比特

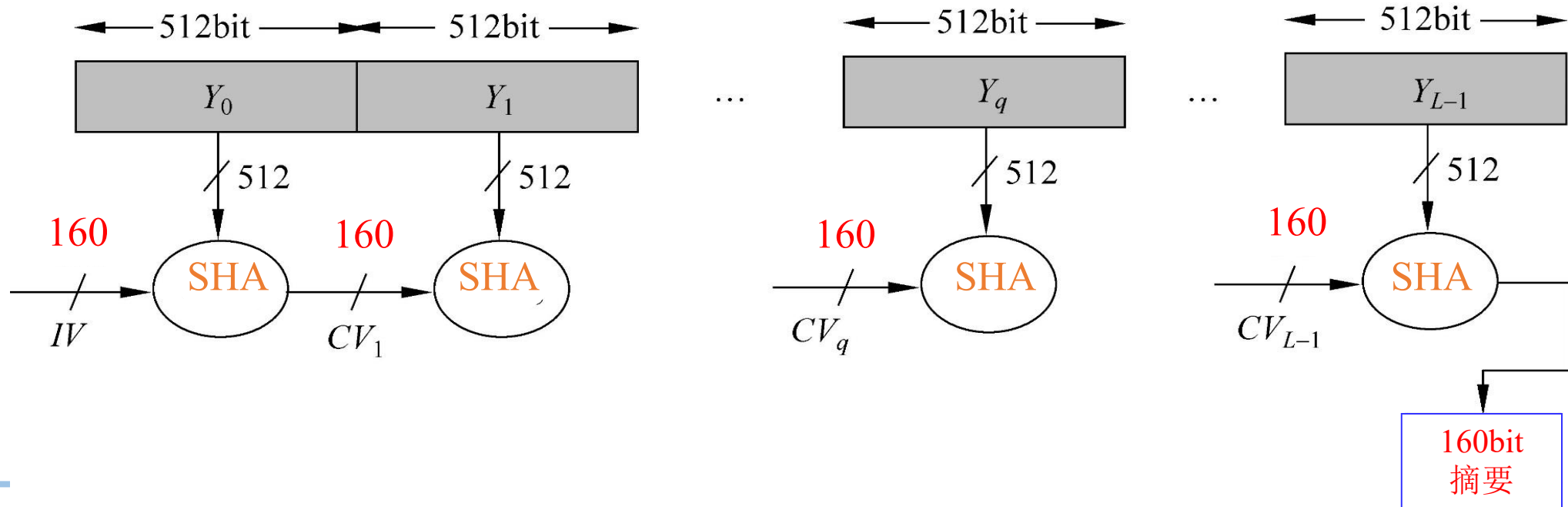
$$K < 2^{64} \text{ bit}$$

填充 (1 到 512bit) 消息长度 ($K \bmod 2^{64}$)

K

$$L \times 512 \text{ bit} = N \times 32 \text{ bit}$$

$K \text{ bit}$



算法处理步骤--- ①对消息填充

- 对消息填充，使得其比特长在模512下为448，即填充后消息的长度为512的某一倍数减64，留出的64比特备第②步使用。
- 步骤①是必需的，即使消息长度已满足要求，仍需填充。例如，消息长为448比特，则需填充512比特，使其长度变为960，因此填充的比特数大于等于1而小于等于512。
- 填充方式是：第1位为1，其后各位皆为0。

算法处理步骤--- ②附加消息的长度

- 留出的64比特用来表示消息被填充前的长度。如果消息长度大于 2^{64} ，则以 2^{64} 为模数取模。
- **big-endian模式**是指数据的高字节保存在内存的低地址中，反之为**little-endian模式**

算法处理步骤-- ③对MD缓冲区初始化

- 使用160比特长的缓冲区存储中间结果和最终杂凑值。

缓冲区为5个32比特以big-endian方式存储数据的寄存器(A, B, C, D, E)
初始值分别为A=67452301,
B=EFCDAB89,
C=98BADCFB,
D=10325476,
E=C3D2E1F0。

算法处理步骤-- ④ 以分组为单位对消息进行处理

- 每一分组 Y_q 都经一压缩函数处理，压缩函数由4轮处理过程构成，每一轮又由20步迭代组成。
- 第4轮的输出（即第80步迭代的输出）再与第1轮的输入 CV_q 相加，以产生 CV_{q+1} ，其中加法是缓冲区5个字中的每一个字与 CV_q 中相应的字模 2^{32} 相加。

算法处理步骤-- ⑤ 输出消息

- 输出消息的 L 个分组都被处理完后，最后一个分组的输出即为160比特的消息摘要。

总结---③到⑤的处理过程

$$CV_0 = IV;$$

$$CV_{q+1} = \text{SUM}_{32}(CV_q, \text{ABCDE}_q);$$

$$\text{MD} = CV_L$$

IV: 缓冲区ABCDE的初值。

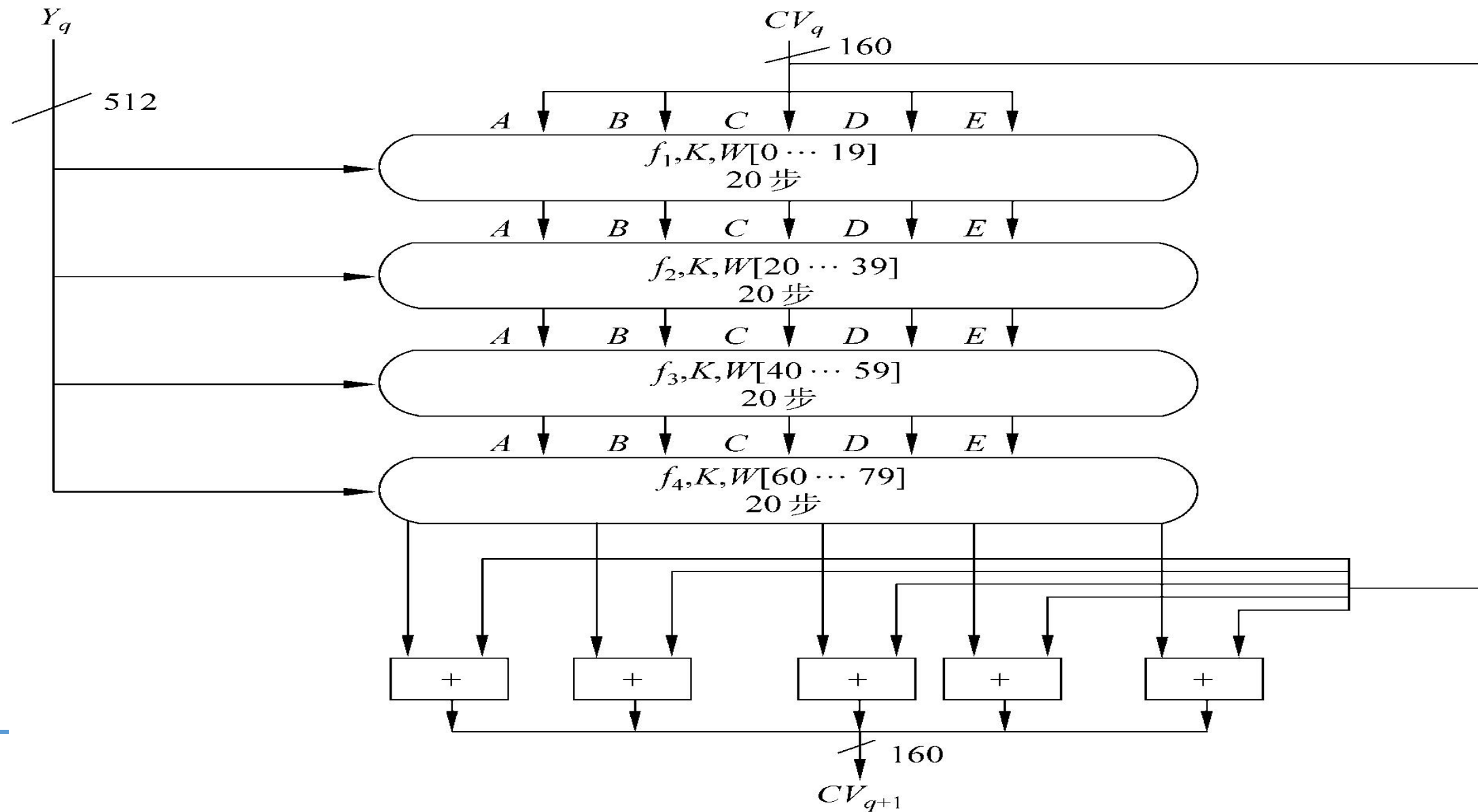
ABCDE_q : 第q个消息分组经最后一轮处理过程处理后的输出

L: 消息(包括填充位和长度字段)的分组数

SUM_{32} : 对应字的模 2^{32} 加法

MD: 最终的摘要值。

SHA的分组处理框图



SHA的压缩函数

- SHA的压缩函数由4轮处理过程组成，每轮处理过程20步迭代运算组成，每一步迭代运算的形式为

$$A, B, C, D, E \leftarrow (E + f_t(B, C, D) + CLS_s(A) + W_t + K_t), A, CLS_{30}(B), C, D$$

A, B, C, D, E: 缓冲区的5个字

t: 迭代的步数 ($0 \leq t \leq 79$)

$f_t(B, C, D)$: 第t步迭代使用的基本逻辑函数

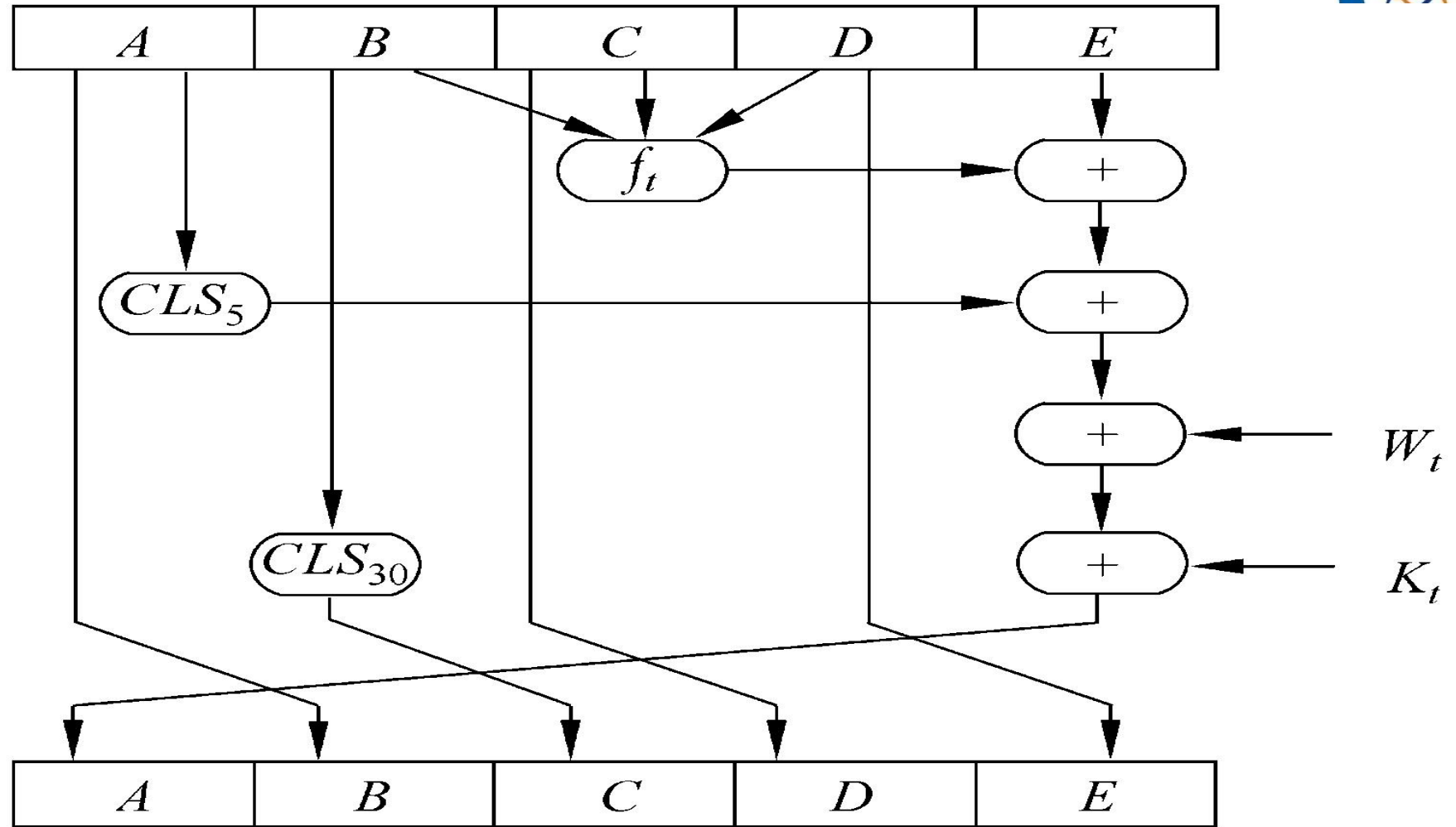
CLSs: 左循环移s位

W_t : 由当前512比特长的分组导出的一个32比特长的字

K_t : 加法常量

+: 模 2^{32} 加法。

一步迭代示意图



$$A, B, C, D, E \leftarrow (E + f_t(B, C, D) + CLS_5(A) + W_t + K_t), A, CLS_{30}(B), C, D$$

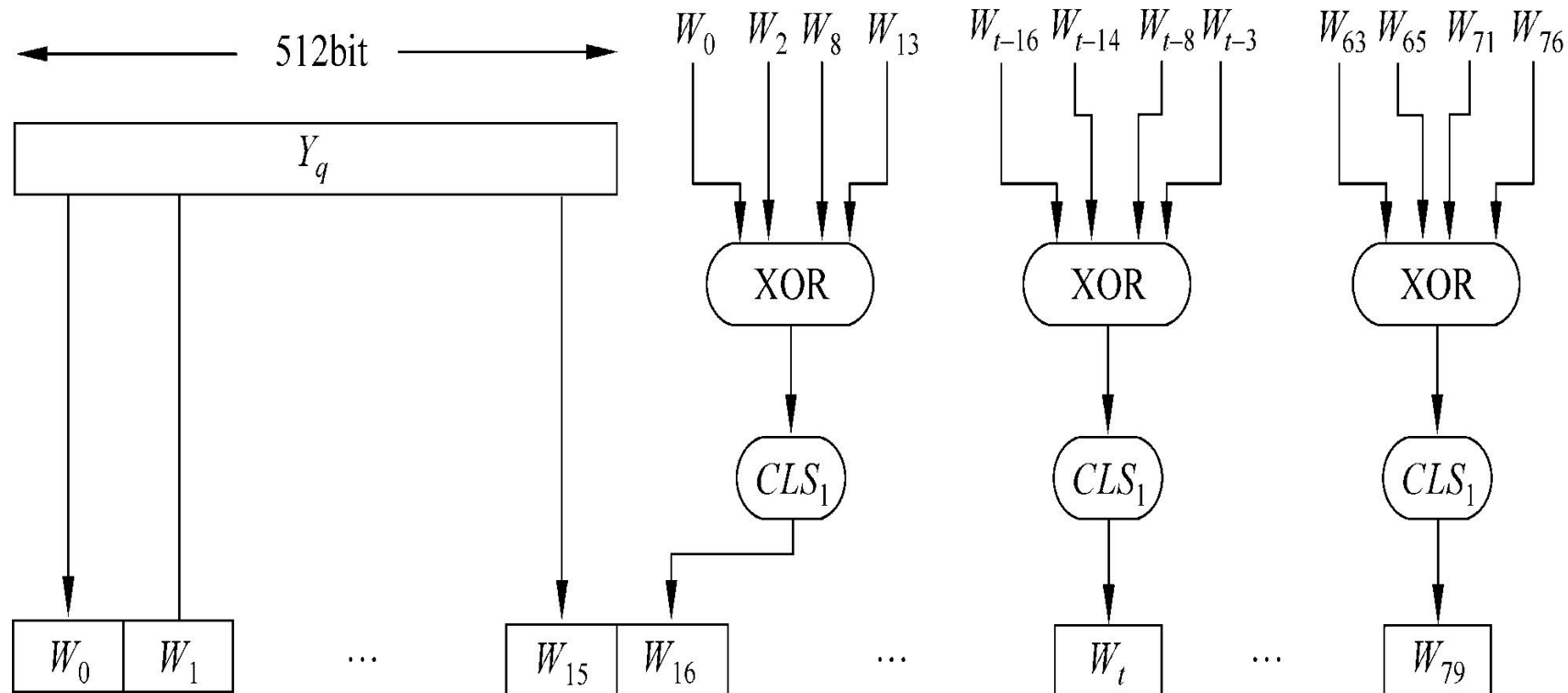
基本逻辑函数 f_t

迭代的步数	函数名	定义
$0 \leq t \leq 19$	$f_1 = f_t(B, C, D)$	$(B \wedge C) \vee (\overline{B} \wedge D)$
$20 \leq t \leq 39$	$f_2 = f_t(B, C, D)$	$B \oplus C \oplus D$
$40 \leq t \leq 59$	$f_3 = f_t(B, C, D)$	$(B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$
$60 \leq t \leq 79$	$f_4 = f_t(B, C, D)$	$B \oplus C \oplus D$

- 基本逻辑函数的输入为3个32比特的字，输出是一个32比特的字。表中 $\wedge$ ， $\vee$ ， $\neg$ ， $\oplus$ 分别是与、或、非、异或4个逻辑运算。

SHA分组处理所需的80个字的产生过程

$$W_t = CLS_1(W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3})$$



常量字 K_t

- 常量字 $K_0, K_1, \dots, K_{79}$, 如果以16进制给出。它们如下:

$$K_t = 0x5A827999 \quad (0 \leq t \leq 19)$$

$$K_t = 0x6ED9EBA1 \quad (20 \leq t \leq 39)$$

$$K_t = 0x8F1BBCDC \quad (40 \leq t \leq 59)$$

$$K_t = 0xCA62C1D6 \quad (60 \leq t \leq 79).$$

SHA系列

1、SHA系列Hash函数

- **SHA 系列标准Hash函数**是由美国标准与技术研究所(NIST)组织制定的。
- 1993年公布了**SHA-0** (FIPS PUB 180), 后发现不安全。
- 1995年又公布了**SHA-1** (FIPS PUB 180-1)。
- 2002年又公布了**SHA-2** (FIPS PUB 180-2):
 - SHA-256
 - SHA-384
 - SHA-512
- 2005年中国王小云给出一种攻击**SHA-1**的方法, 用 2^{69} 操作找到一个强碰撞, 以前认为是 2^{80} 。

SHA参数比较

SHA参数比较

	SHA-1	SHA-256	SHA-384	SHA-512
Hash码长度	160	256	384	512
消息长度	$<2^{64}$	$<2^{64}$	$<2^{128}$	$<2^{128}$
分组长度	512	512	1024	1024
字长度	32	32	64	64
迭代步骤数	80	64	80	80
安全性	80	128	192	256

注:1、所有的长度以比特为单位。

2、安全性是指对输出长度为n比特hash函数的生日攻击产生碰撞的工作量大约为 $2^{n/2}$

王小云院士

- 王小云，清华大学高等研究院“杨振宁讲座”教授，中国科学院院士，国际密码协会会士(IACR Fellow)，中国密码学会副理事长。
- 国际密码学会（IACR）公布了2020年“**最具时间价值奖**”（Test-of-Time Awards）的获奖论文，王小云院士以第一作者身份在2005年发表于美密会（Crypto）上的论文，因在哈希函数分析上取得的突破性成果而获奖。
 - “最具时间价值奖”是国际密码协会（IACR）自2019年起设立的，该奖项每年会从15年前的三大密码学会会议欧密会（Eurocrypt）、美密会（Crypto）和亚密会（Asiacrypt）中各选出一篇论文，以表彰他们对密码学领域产生的持久影响和重要贡献。
- 王小云院士2020年1月获得“真实世界密码学奖”
 - （The Levchin Prize for Real-World Cryptography）
 - “真实世界密码学奖”是2015年互联网企业家马克斯·莱文奇恩（Max Levchin）创立的。该奖项旨在表彰对真实世界密码学作出重大贡献，和在密码学实践及系统应用中取得了具有重大影响的研究进展。每年最多颁发两个奖项。





现代密码学

SHA3密码杂凑函数

信息与软件工程学院

SHA-3算法

- 国际标准
 - MD5, SHA0, SHA1: 基本上都被破译了 (王小云)
 - SHA2 (FIPS PUB 180-2标准), 对SHA1进行了改进, 包括3个算法SHA-256, SHA384, SHA-512, 仅是过渡算法。
 - SHA-3候选算法, 最后一轮五个候选算法是:
 - 源自瑞士的**BLAKE**
 - **Graz**理工大学和丹麦理工大学合作的**Grøstl**
 - 新加坡华裔信息安全专家伍宏军的**JH**(已经发现了些问题)
 - **Joan Daemen**为主要成员的**Keccak**
 - **Bruce Schneier**为负责人的**Skein**。
 - 2012年9月美国国家标准技术研究协会 (NIST) 宣布了SHA-3算法竞赛的获胜者——Keccak (/ˈkɛtʃæk/)
-

SHA-3时间点

- 2007年11月： 宣布竞争
- 2008年10月： 64 入围
- 2008年12月： 51 个进入第 1轮
- 2009年7月： 14 个进入第2轮
- 2010年12月：5 个进入第3轮 （BLAKE, Grøstl, JH, Keccak, Skien）
- 2011年1月： 最后的提交和公众评论
- 2012年9： SHA-3宣布

▶**Keccak获胜**

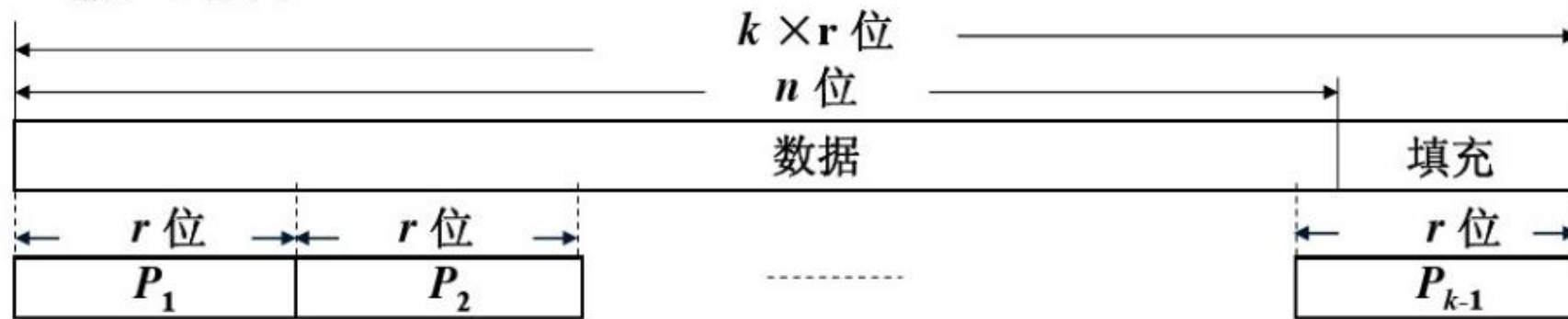
▶**作者： Joan Daemen*, Guido Bertoni, Michaël Peeters, Gilles Van Assche**

SHA-3结构

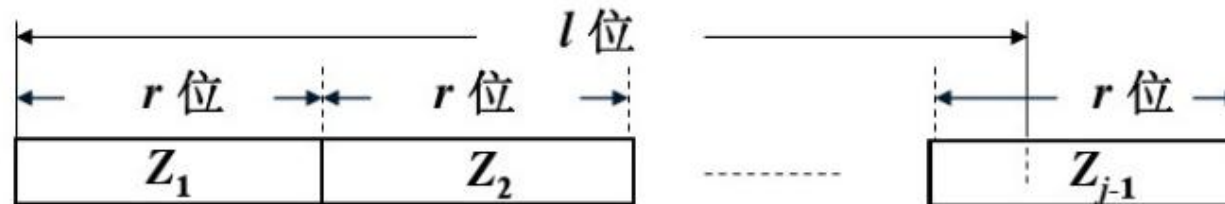
- **SHA-3**的结构仍属于Merkle提出的一般结构。
- 最大的创新点是采用了一种被称为**海绵结构**的新的迭代结构。海绵结构又称为海绵函数。
- 在海绵函数中，输入数据被分为固定长度的数据分组。每个分组逐次作为迭代的输入，同时上轮迭代的输出也反馈至下轮的迭代中，最终产生输出Hash码。
- 海绵函数允许**输入长度和输出长度都可变**，具有灵活的性。
- 海绵函数能够用于设计**Hash函数（固定输出长度）**、**伪随机数发生器**，以及其他密码函数。

SHA3的输入输出结构

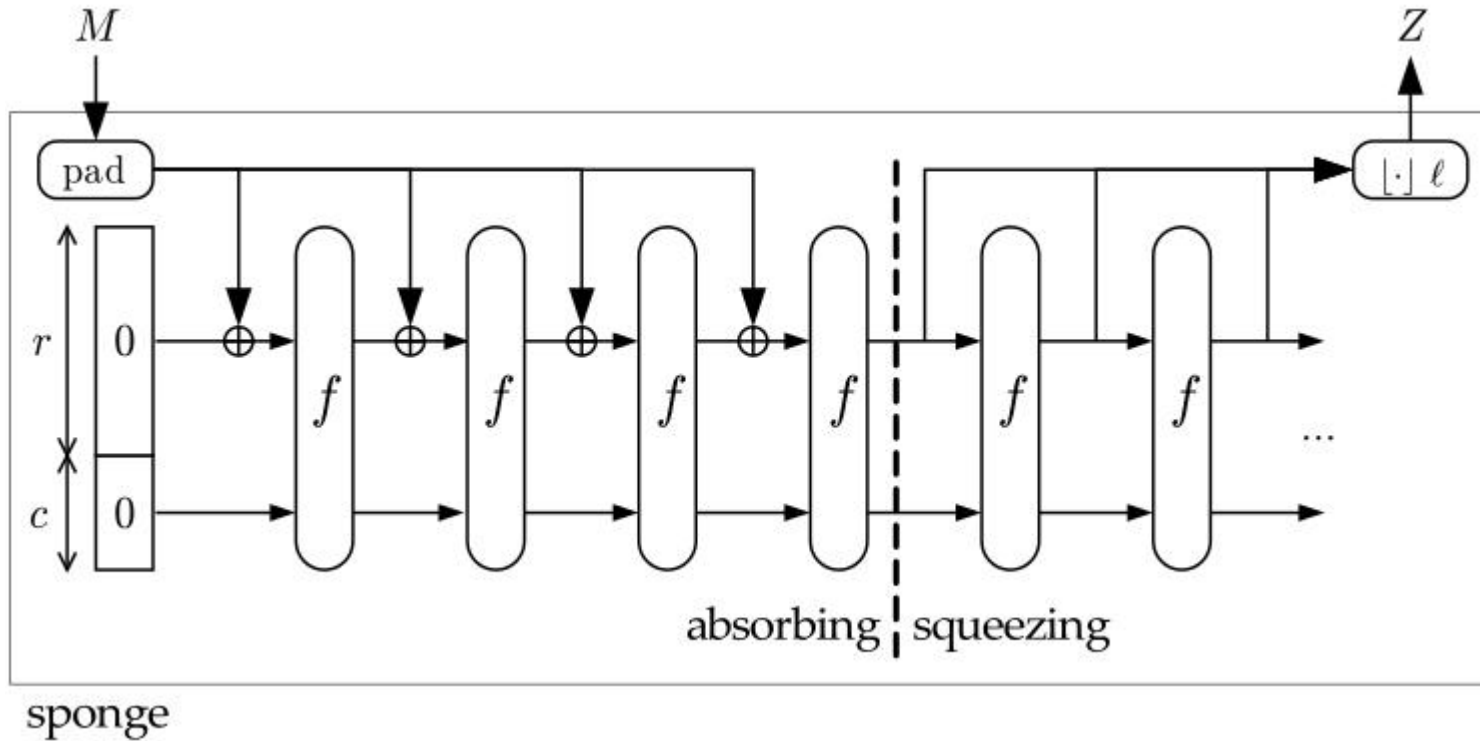
● 输入结构



● 输出结构

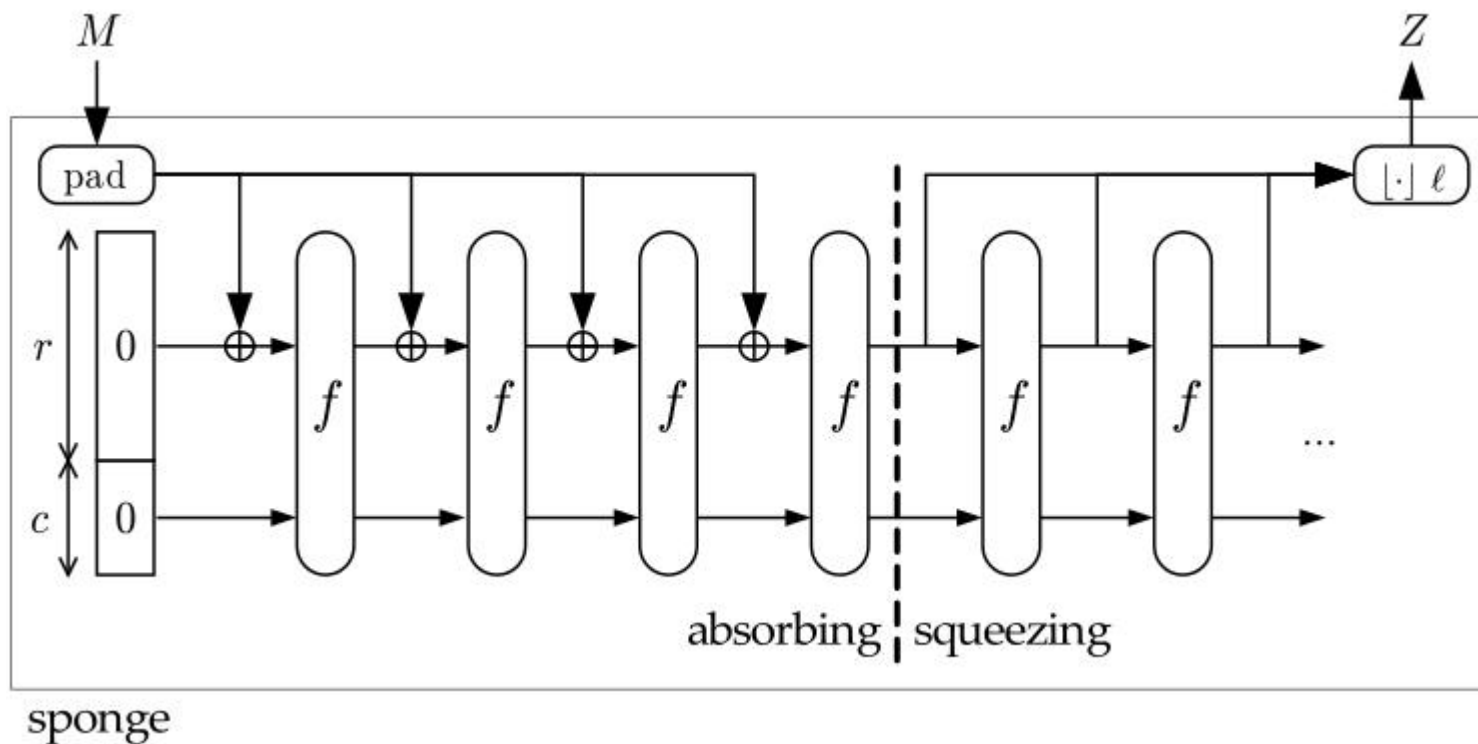


Keccak概述



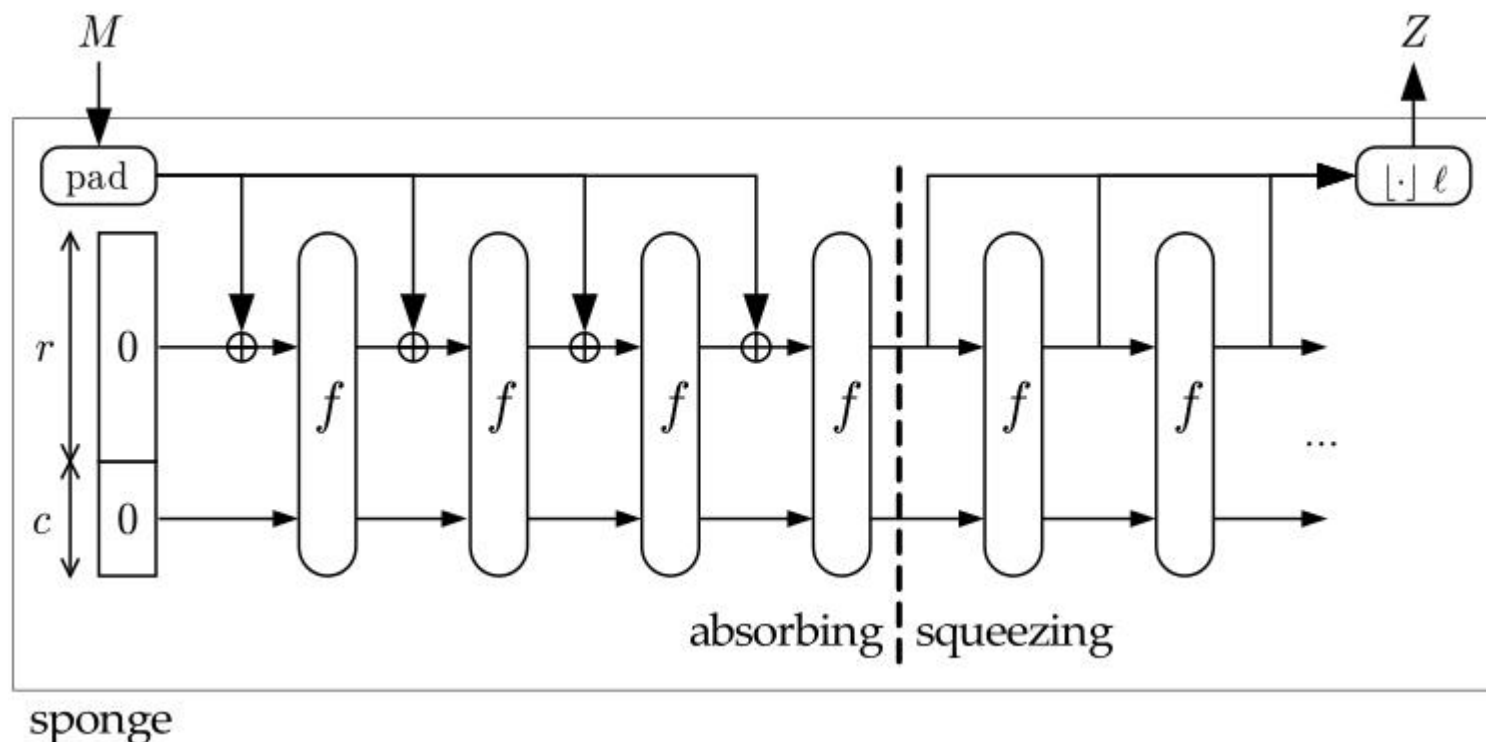
$b = 25 \times 2^\ell = r + c$, r 为rate效率, c 为capacity容量。数据 M 的填充规则: 在数据 M 结尾填充1, 然后填充数量最少的0, 最后在结尾填充1, 使得填充后的数据长度为数据块长度 r 的倍数。填充数据的长度最短为2位, 最长为 $r+1$ 位。

Keccak概述



- 吸收过程：
1. **state S**被初始化为0
 2. 对输入进行填充
 3. 将第一组 r 个输入bit与 r 进行异或
 4. **S**被 $f(S)$ 取代
 5. 下一个 r 个输入比特与 r 进行异或
 6. **S**被 $f(S)$ 取代 ...循环直至所有的 r 数据块被计算完成

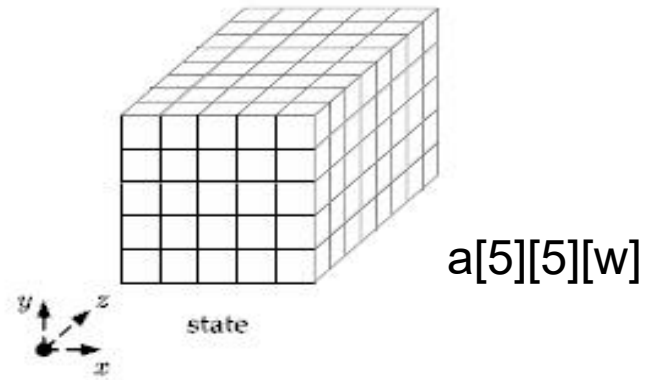
Keccak概述



- 挤压过程：1. 将所得运算结果 S 截取 r 位，赋值给 Z ；
- 2. 如果 r 小于输出结果 n ，对 S 进行迭代运算；
- 3. 将迭代运算结果截取 r 位级联到 Z 后面，循环进行迭代和级联直到 Z 的长度大于 n ；
- 4. 截取 Z 的前 n 位，所得结果即为Keccak的运算结果。



keccak概述

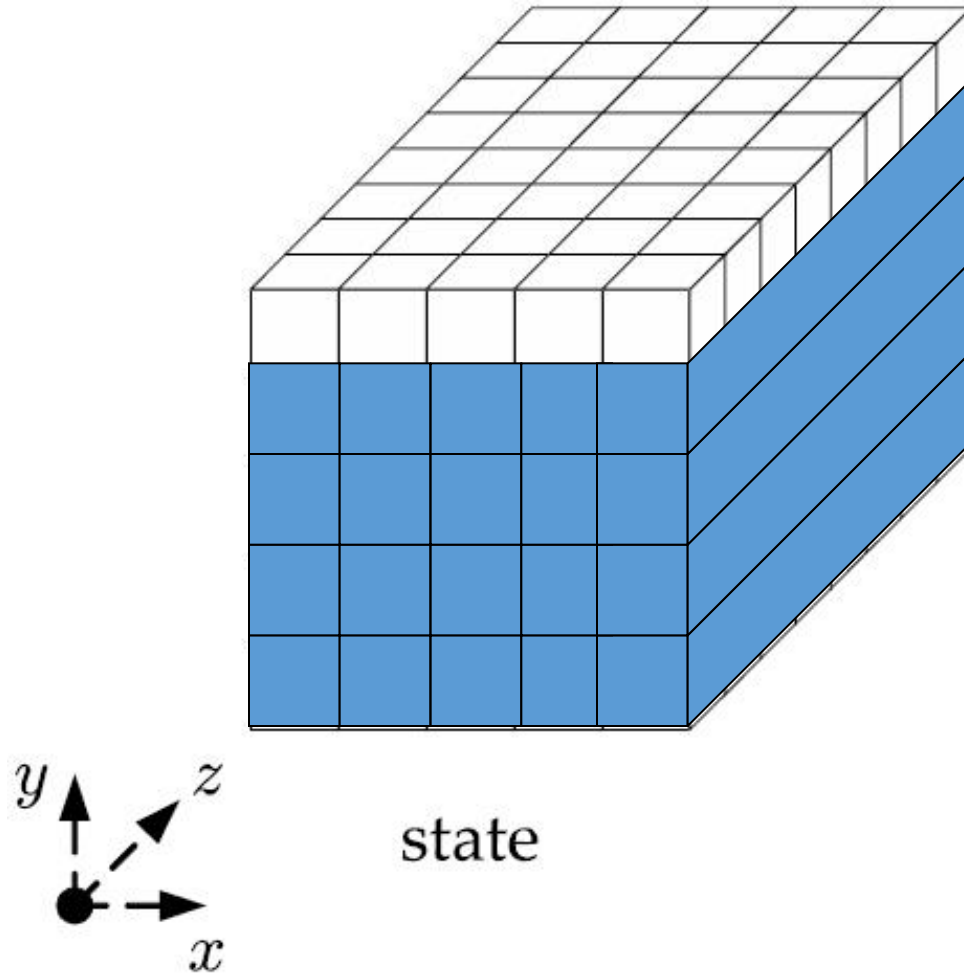


$$b=25 \times 2^\ell$$

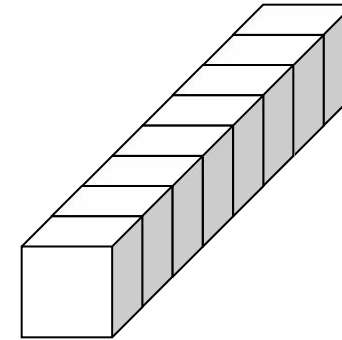
$$w=2^\ell$$



state



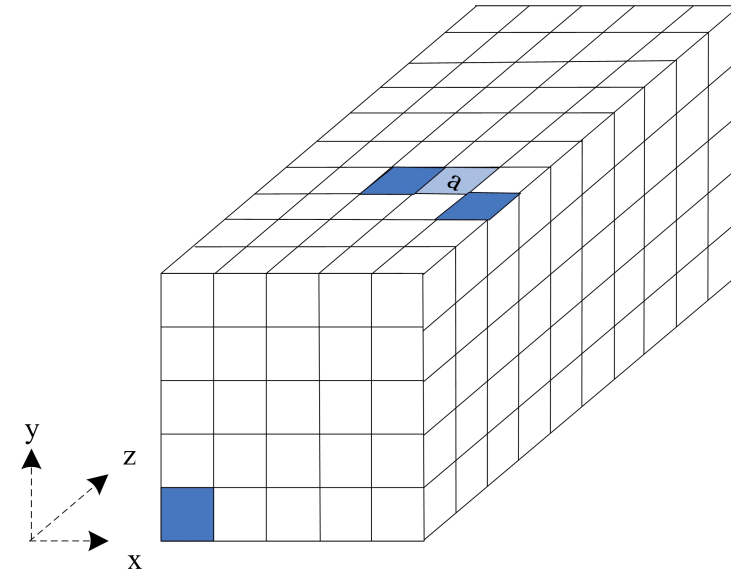
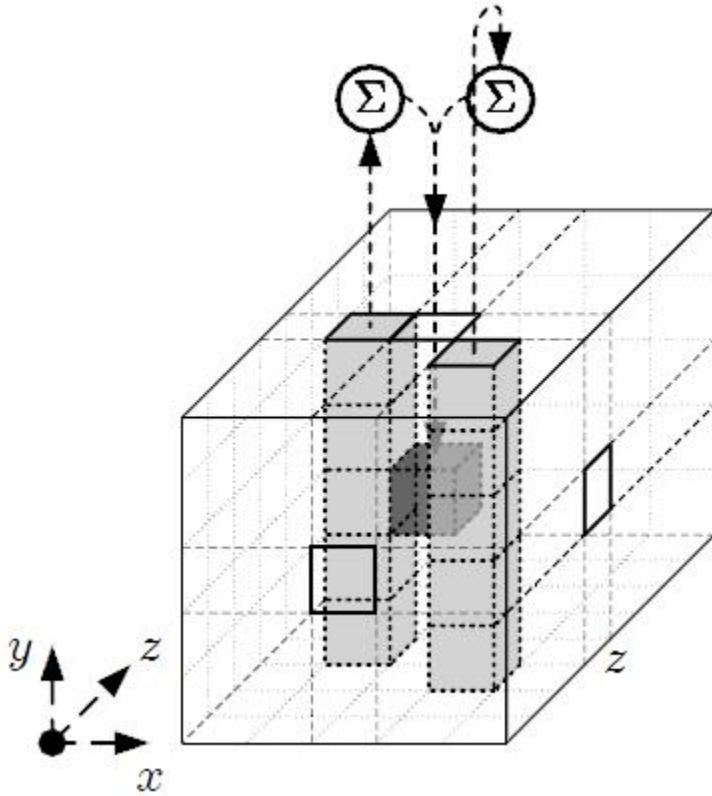
$$S[w(5y+x)+z]=a[x][y][z]$$





$f[b]: \theta$

$$\theta : a[x][y][z] \leftarrow a[x][y][z] + \sum_{y'=0}^4 a[x-1][y'][z] + \sum_{y'=0}^4 a[x+1][y'][z-1]$$



θ 是线性的，以扩散为目的。

$f[b]: \rho$

$$\rho : a[x][y][z] \leftarrow a[x][y][z-(t+1)(t+2)/2] \quad \text{满足} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}^t \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix}$$

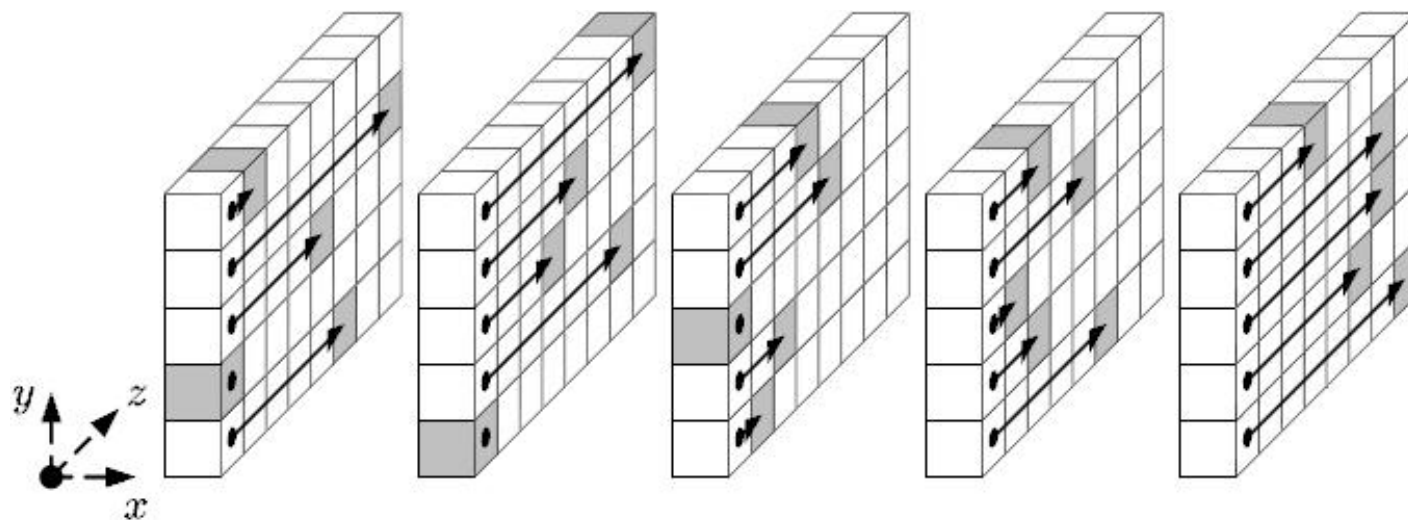
当 $x=y=0$ 时, $t=-1$ 。 $T_t = t(t+1)/2$

用不同的三角形数对**slice**上的**25**个数进行比特旋转（**lane**方向循环移位）。 (**0,1,3,6,10,15,21,28,36...**)

	x=3	x=4	x=0	x=1	x=2
y=2	153	231	3	10	171
y=1	55	276	36	300	6
y=0	28	91	0	1	190
y=4	120	78	210	66	253
y=3	21	136	105	45	15



$f[b]: \rho$

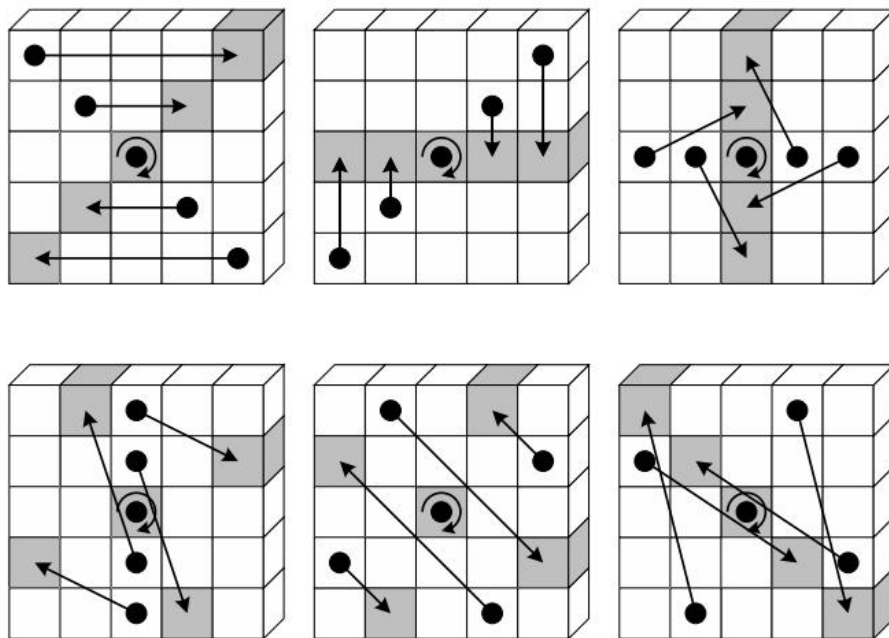


➔ ρ : **slice**内部的转换，使每个**slice**之间的离散加快



$f[b]:$

$\pi : a[x][y] \leftarrow a[x'][y']$ 其中

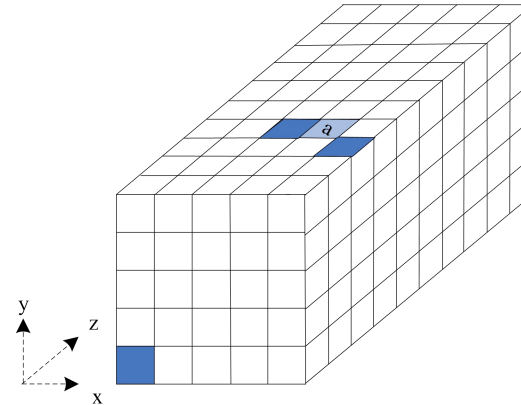
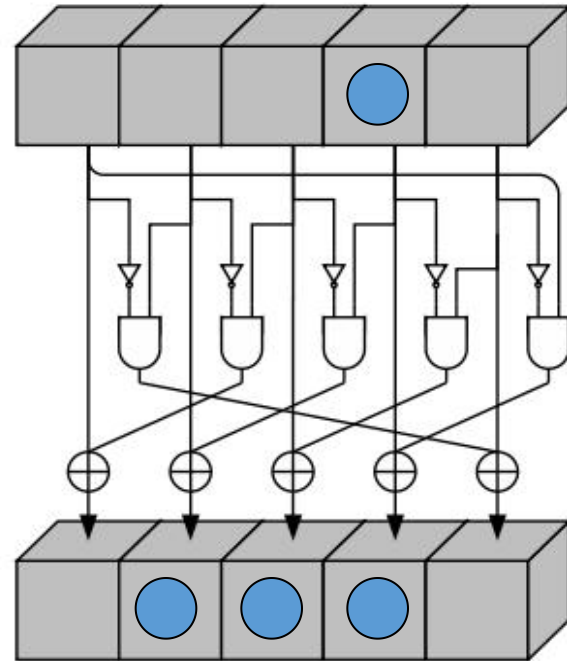


- $x=y=0$ 位于平面中心
- π : 对lane进行映射, 三维数组高度离散, z 轴有变换不变性



$f[b]: x$

$$\chi : a[x] \leftarrow a[x] + (a[x+1] + 1)a[x+2]$$



χ : 唯一的非线性映射，实现简单的非线性传播。

该函数与 θ 相互作用，使每一轮迭代的每个输入比特都潜在的影响了31个输出比特

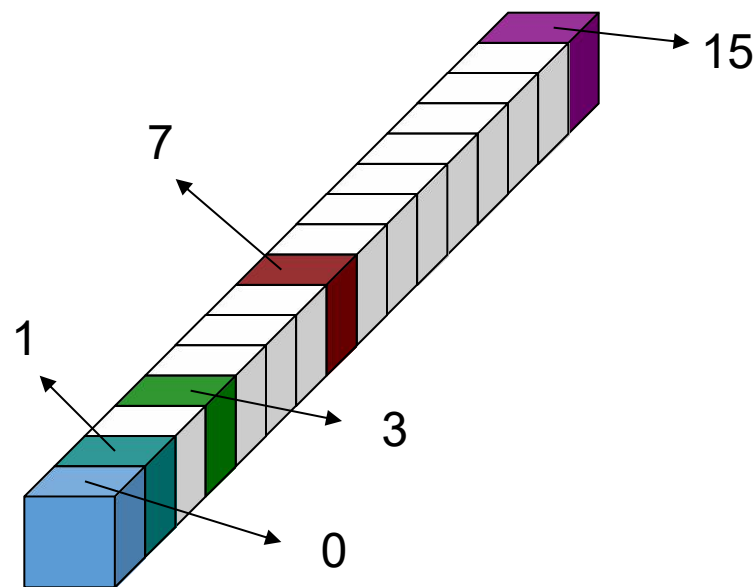
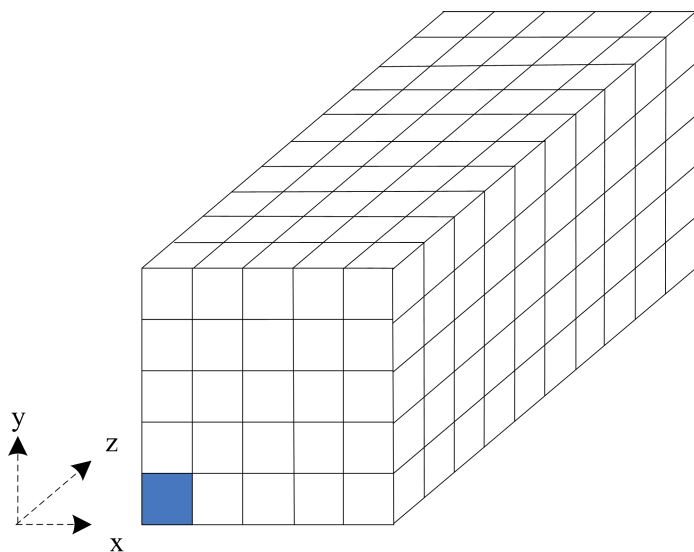


$f[b]: \tau$

$$t : a \leftarrow a + RC[i_r]$$

$$RC[i_r][0][0][2^j - 1] = rc[j + 7i_r], 0 \leq j \leq l$$

$$rc[t] = \left(x^t \bmod (x^8 + x^6 + x^5 + x^4 + 1) \right) \bmod x \text{ in } GF(2)[x]$$





$f[b]$



sponge函数中， f 对 θ 、 ρ 、 π 、 χ 和 τ 五个过程进行迭代运算，而迭代次数由 $n_r=12+2l$ 决定。

$$f[b] = \tau \circ \chi \circ \pi \circ \rho \circ \theta$$

参数选择

- r 决定了每个数据块中数据的长度, c 决定了这个函数的安全级别。
- $c=2n$. 安全性强度为 2^n

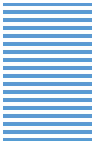
针对 **$b=1600$** , 建议的参数长度:

$$n = 224: \lfloor \text{Keccak}[r = 1152, c = 448] \rfloor_{224}$$

$$n = 256: \lfloor \text{Keccak}[r = 1088, c = 512] \rfloor_{256}$$

$$n = 384: \lfloor \text{Keccak}[r = 832, c = 768] \rfloor_{384}$$

$$n = 512: \lfloor \text{Keccak}[r = 576, c = 1024] \rfloor_{512}$$

A decorative graphic consisting of several horizontal blue bars of varying lengths, stacked vertically, is positioned to the left of the title.

SHA-3的特点

- 安全性

- 可以抵御对Hash函数的现有攻击。
- 到目前为止，没有发现它有严重的安全弱点。

- 灵活性

- 可选参数配置，能够适应Hash函数的各种应用。

- 高效性

- 设计简单，软硬件实现方便。在效率方面，它是高效的。

- 尚未广泛应用，需要经过实践检验！



现代密码学

SM3密码杂凑函数

信息与软件工程学院



中华人民共和国密码行业标准

GM/T 0004—2012

SM3 密码杂凑算法

SM3 cryptographic hash algorithm

2012-03-21 发布

2012-03-21 实施

国家密码管理局 发布

SM3密码杂凑算法

- SM3是中国国家密码管理局颁布的中国商用公钥密码标准算法，它是一类密码杂凑函数，可用于数字签名及验证、消息认证码生成及验证、随机数生成。
 - 标准起草人：王小云、李峥、于红波、张超、罗鹏、吕述望
 - 2012年3月，成为中国商用密码标准（GM/T 0004-2012）
 - 2016年8月，成为中国国家密码标准（GB/T 32905-2016）
 - 2018年11月22日，含有我国SM3杂凑密码算法的ISO/IEC 10118 – 3 : 2018 《信息安全技术杂凑函数第3部分:专用杂凑函数》最新一版(第4版)由国际标准化组织(ISO)发布，SM3算法正式成为国际标准。
-

SM3密码杂凑算法的描述

算法的输入数据长度为 l 比特, $l < 2^{64}$, 输出哈希值长度为**256**比特。

1. 常数与函数

(1) 常数

初始值

$IV = 7380166F4914B2B9$

$172442D7DA8A0600$

$A96F30BC163138AA$

$E38DEE4DB0FB0E4E$

常量

$$T_j = \begin{cases} 79CC4519, & 0 \leq j \leq 15 \\ 7A879D8A, & 16 \leq j \leq 63 \end{cases}$$

SM3密码杂凑算法的描述

(2) 函数 式中 X, Y, Z 为32位字, $\wedge, \vee, -, \oplus$ 分别是逻辑与、逻辑
布尔函数: 或、逻辑非和逐比特异或运算

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), & 16 \leq j \leq 63 \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\bar{X} \wedge Z), & 16 \leq j \leq 63 \end{cases}$$

置换函数:

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17);$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23).$$

混淆

扩散

式中 X 为32位字, 符号 $a \lll n$ 表示把 a 循环左移 n 位。

SM3密码杂凑算法的描述

2. 算法描述

算法对数据首先进行填充，再进行迭代压缩后生成哈希值。

(1) 填充并附加消息的长度

- 对消息填充的目的是使填充后的数据长度为**512的整数倍**。
- 设消息 m 的长度为 l 比特。首先将比特“1”添加到 m 的末尾，再添加 k 个“0”，其中 k 满足

$$l+1+k=448 \bmod 512$$

- 然后再添加一个64位比特串，该比特串是长度 l 的二进制表示。

举例：消息为011000010110001001100011，其长度为 $l=24$ ，填充后的

比特串为

011000010110001001100011**1**
423个064比特
00...0000...011000

SM3密码杂凑算法的描述

(2) 迭代压缩

将填充后的消息 m' 按512比特进行分组得 $m' = B^{(0)}B^{(1)} \dots B^{(L-1)}$ 对 m' 按下列方式迭代压缩:

$$\text{FOR } i=0 \text{ to } L-1 \quad V^{(i+1)} = CF(V^{(i)}, B^{(i)})$$

其中 CF 是压缩函数, $V^{(0)}$ 为 256比特初始值 IV , $B^{(i)}$ 为填充后的消息分组, 迭代压缩的结果为 $V^{(L)}$, $V^{(L)}$ 即为消息 m 的哈希值。

SM3密码杂凑算法的描述

(3) 消息扩展

在对消息分组 $B^{(i)}$ 进行迭代压缩之前，首先对其进行消息扩展，步骤如下：

① 消息分组 $B^{(i)}$ 划分为**16**个字 $W_0, W_1, \dots, W_{15}$ 。

② FOR $j=16$ to 67

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$$

③ FOR $j=0$ to 63

$$W'_j = W_j \oplus W_{j+4}$$

$B^{(i)}$ 经消息扩展后得到**132**个字 $W_0, W_1, \dots, W_{67}, W'_0, W'_1, \dots, W'_{63}$ 。

SM3密码杂凑算法的描述

(4) 压缩函数

设 A, B, C, D, E, F, G, H 为字寄存器, SS_1, SS_2, TT_1, TT_2 为中间变量, 压缩函数 $V^{(i+1)} = CF(V^{(i)}, B^{(i)}) (0 \leq i \leq n-1)$ 的计算过程如下:

$$ABCDEFGH = V^{(i)}$$

FOR $j=0$ to 63

$$SS_1 \leftarrow ((A \lll 12) + E + (T_j \lll j)) \lll 7;$$

$$SS_2 = SS_1 \oplus (A \lll 12);$$

$$TT_1 = FF_j(A, B, C) + D + SS_2 + W'_j;$$

$$TT_2 = GG_j(E, F, G) + H + SS_1 + W_j;$$

$$D = C;$$

$$C = B \lll 9;$$

$$B = A;$$

$$A = TT_1;$$

$$H = G;$$

$$G = F \lll 19;$$

$$F = E;$$

$$E = P_0(TT_2)$$

ENDFOR

$$V^{(i+1)} = ABCDEFGH \oplus V^{(i)}$$

其中 $+$ 为模 2^{32} 加运算, 字的存储为大端格式



SM3密码杂

$$ABCDEFGH = V^{(i)}$$

FOR $j=0$ to 63

$$SS_1 \leftarrow ((A \lll 12) + E + (T_j \lll j)) \lll 7;$$

$$SS_2 = SS_1 \oplus (A \lll 12);$$

$$TT_1 = FF_j(A, B, C) + D + SS_2 + W'_j;$$

$$TT_2 = GG_j(E, F, G) + H + SS_1 + W_j;$$

$$D = C;$$

$$C = B \lll 9;$$

$$B = A;$$

$$A = TT_1;$$

$$H = G;$$

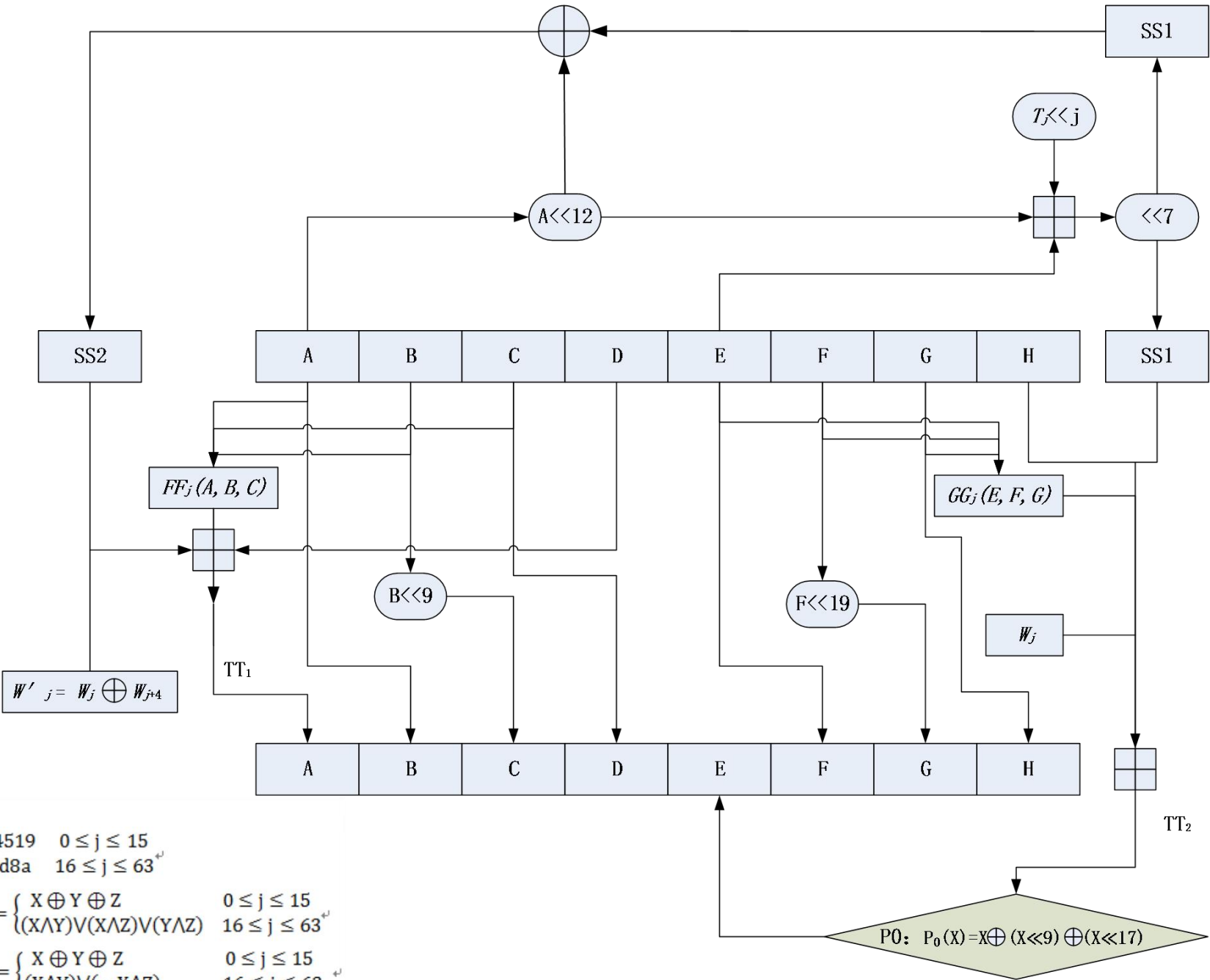
$$G = F \lll 19;$$

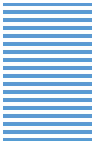
$$F = E;$$

$$E = P_0(TT_2)$$

ENDFOR

$$V^{(i+1)} = ABCDEFGH \oplus V^{(i)}$$



A decorative blue horizontal bar with a series of parallel lines is positioned on the left side of the slide.

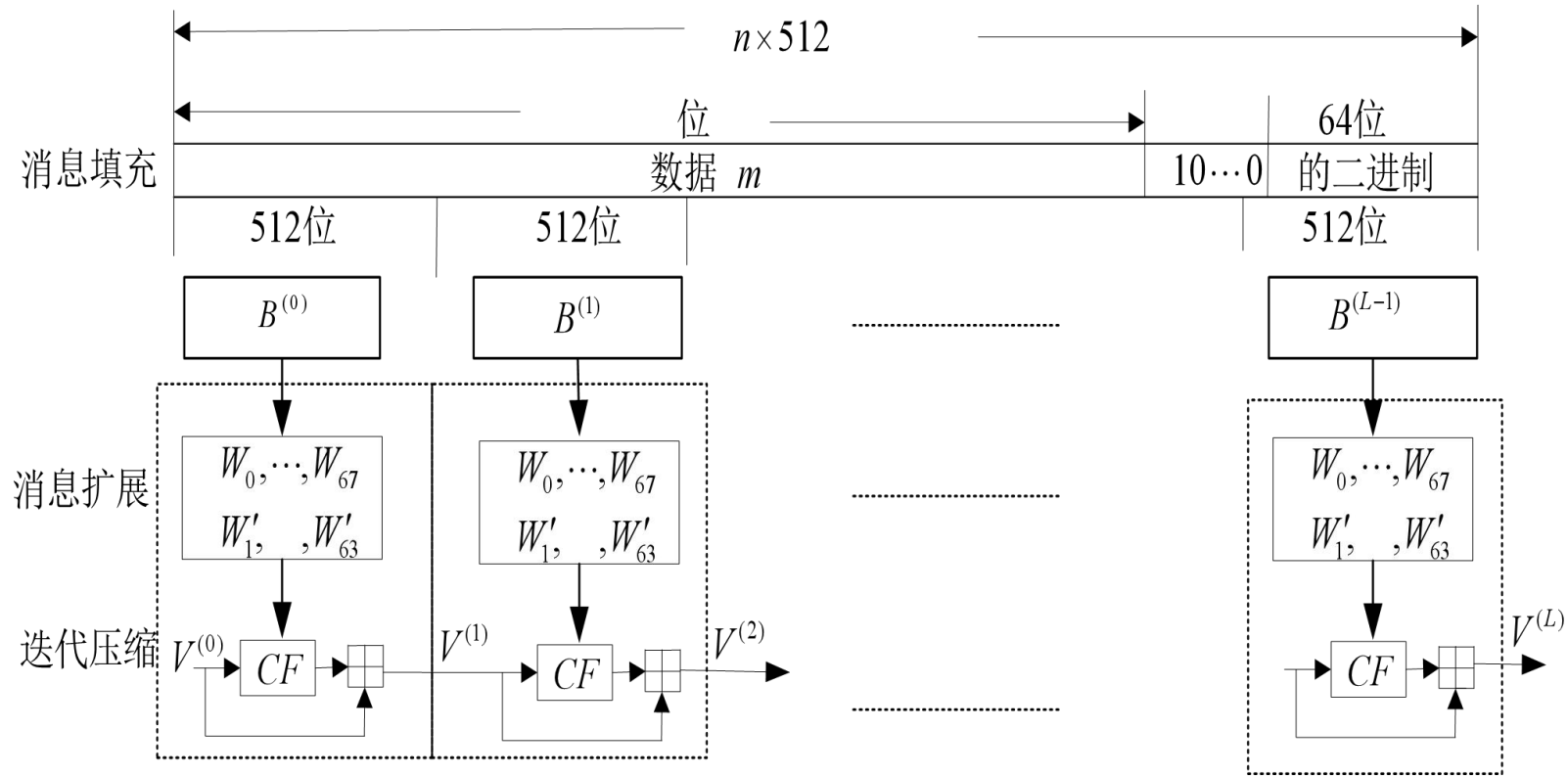
SM3密码杂凑算法的描述

(5) 输出哈希值

$$ABCDEFGH = V^{(L)}$$

输出 256 比特的哈希值 $y = ABCDEFGH$ 。

SM3密码杂凑算法的描述



SM3 产生消息哈希值的处理过程

SM3哈希算法的安全性

- 压缩函数是哈希函数安全的关键
 - SM3的压缩函数 CF 中的布尔函数 $FF_j(X,Y,Z)$ 和 $GG_j(X,Y,Z)$ 是非线性函数，经过循环迭代后提供混淆作用
 - 置换函数 $P_0(X)$ 和 $P_1(X)$ 是线性函数，经过循环迭代后提供扩散作用。
 - 再加上 CF 中的其他运算的共同作用，压缩函数 CF 具有很高的安全性，从而确保SM3具有很高的安全性。



算法比较

王小云, 于红波. SM3密码杂凑算法[J]. 信息安全研究, 2016(11).

表 2 SM3 密码杂凑算法和其他标准的 ASIC 实现

算法	面积 (gates)	时钟 /MHz	吞吐量 /Mbps	吞吐量面积比 /(Kbps/gate)
SM3 ^[10]	11 068	216.00	1 619	146.28
SHA-256 ^[11]	15 400	189.75	1 349	87.60
SHA-512 ^[11]	30 747	169.20	1 969	64.04
Whirlpool ^[11]	38 911	101.94	2 485	63.86
SHA-3 ^[12]	56 320	487.80	21 229	376.94

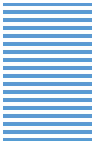
表 5 SM3 密码杂凑算法和其他杂凑标准的最好分析结果

算法	攻击类型	步(轮)数	百分比/%	文献
SM3	碰撞攻击	20	31	[18]
	原像攻击	30	47	[24-25]
	区分器攻击	37	58	[27]
SHA-1	碰撞攻击	80	100	[4,28-29]
	原像攻击	62	77.5	[30]
RIPEMD-128	碰撞攻击	40	62.5	[31]
	原像攻击	36	56.25	[32]
	区分器攻击	64	100	[33]
RIPEMD-160	原像攻击	34	53.12	[34]
	区分器攻击	51	79.68	[35]
SHA-256	碰撞攻击	31	48.4	[36]
	原像攻击	45	70.3	[23]
	区分器攻击	47	73.4	[37]
Whirlpool	碰撞攻击	8	80	[38]
	原像攻击	6	60	[38]
	区分器攻击	10	100	[39]
Stribog	碰撞攻击	7.5	62.5	[40]
	原像攻击	6	50	[41]
KECCAK-256	碰撞攻击	5	20.8	[42]
	原像攻击	2	8	[43]
	区分器攻击	24	100	[44]
KECCAK-512	碰撞攻击	3	12.5	[42]
	区分器攻击	24	100	[44]

现代密码学

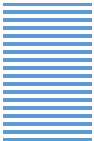
HMAC算法

信息与软件工程学院

A decorative blue horizontal bar with white horizontal stripes is positioned to the left of the title.

HMAC算法

- 以前曾介绍过一个MAC的例子——数据认证算法
 - 该算法反映了传统上构造MAC最为普遍使用的方法，即基于分组密码的构造方法
 - 近年来研究构造MAC的兴趣已转移到基于密码哈希函数的构造方法，这是因为：
 - 哈希函数(如MD5, SHA)软件实现快于分组密码(如DES)的软件实现
 - 哈希函数的库代码来源广泛
 - 哈希函数没有出口限制，而分组密码即使用于MAC也有出口限制
 - 哈希函数并不是为用于MAC而设计的，由于哈希函数不使用密钥，因此不能直接用于MAC
 - 目前已提出了很多将哈希函数用于构造MAC的方法，其中HMAC就是其中之一，已作为RFC2104被公布，并在IPSec和其他网络协议(如SSL)中得以应用
-

A decorative graphic consisting of several horizontal blue bars of varying lengths is positioned to the left of the title.

HMAC的设计目标

- RFC2104列举了HMAC的以下设计目标：
 - ① 可不经修改而使用现有的哈希函数，特别是那些易于软件实现的、源代码可方便获取且免费使用的哈希函数。
 - ② 其中镶嵌的哈希函数可易于替换为更快或更安全的哈希函数。
 - ③ 保持镶嵌的哈希函数的最初性能，不因用于HMAC而使其性能降低。
 - ④ 以简单方式使用和处理密钥。
 - ⑤ 在对镶嵌的哈希函数合理假设的基础上，易于分析HMAC用于认证时的密码强度。
-

- 前两个目标是HMAC被公众普遍接受的主要原因，这两个目标是**将哈希函数当作一个黑盒使用**，这种方式有两个优点：
 - 第一，**哈希函数的实现可作为实现HMAC的一个模块**，这样一来，HMAC代码中很大一块就可事先准备好，无需修改就可使用；
 - 第二，**如果HMAC要求使用更快或更安全的哈希函数，则只需用新模块代替旧模块**，例如用实现SHA的模块代替MD5的模块。
- 最后一条设计目标则是HMAC优于其他基于哈希函数的MAC的一个主要方面，**HMAC在其镶嵌的哈希函数具有合理密码强度假设下，可证明是安全的**



- 算法的输出可如下表示:

$$HMAC_k = H[(K^+ \oplus opad) || H[(K^+ \oplus ipad) || M]]$$

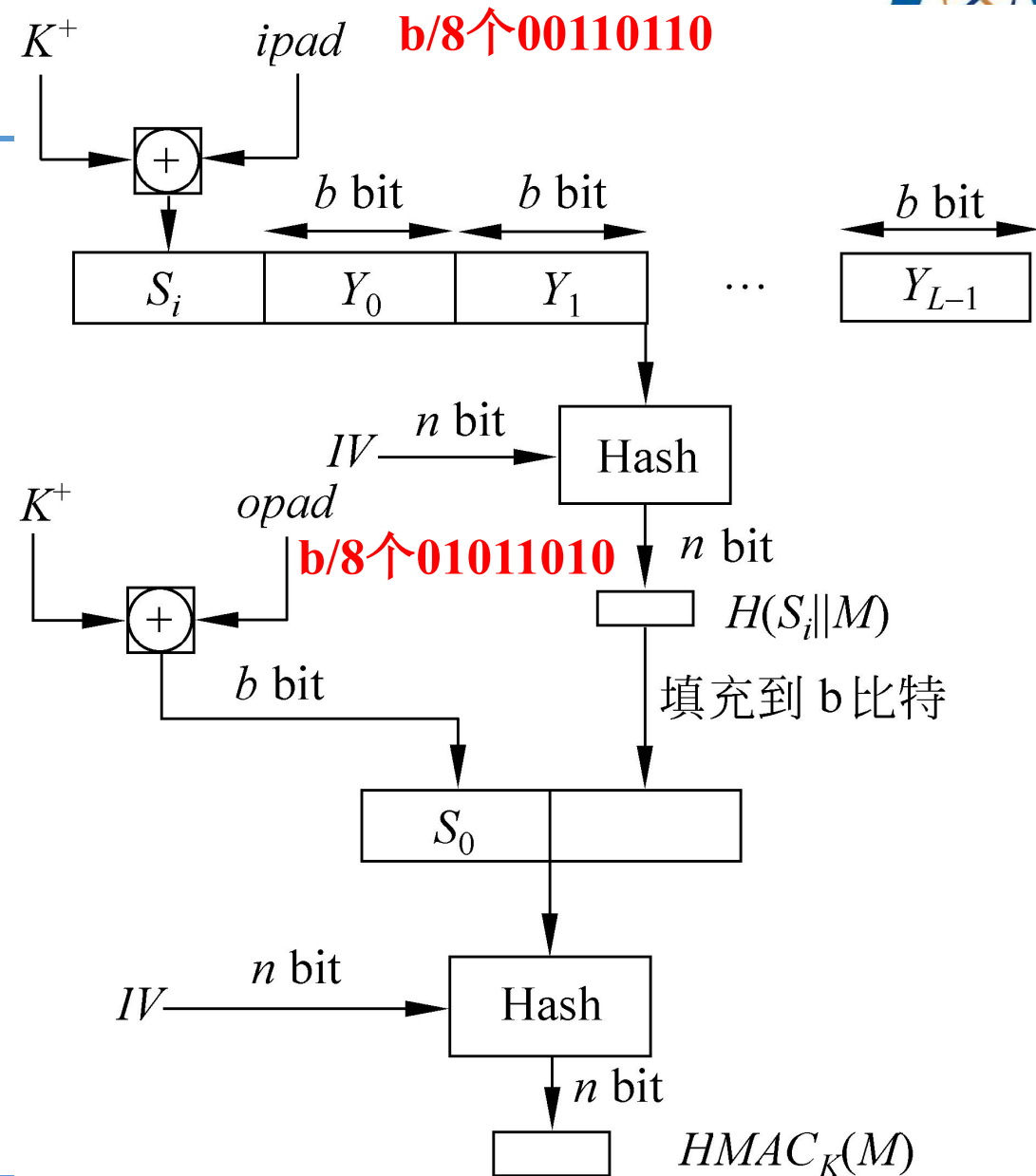


HMAC算法描述

后面经填充0后的
 K ，长度为 b 比特

右图是HMAC算法的运行框图

- H 为嵌入的哈希函数(如MD5、SHA),
- M 为HMAC的输入消息(包括哈希函数所要求的填充位),
- $Y_i (0 \leq i \leq L-1)$ 是 M 的第 i 个分组,
- L 是 M 的分组数,
- b 是一个分组中的比特数,
- n 为由嵌入的哈希函数所产生的哈希值的长度,
- K 为密钥, 如果密钥长度大于 b , 则将密钥输入到哈希函数中产生一个 n 比特长的密钥,
- K^+ 是后面经填充0后的 K , K^+ 的长度为 b 比特,
- $ipad$ 为 $b/8$ 个00110110,
- $opad$ 为 $b/8$ 个01011010。





- 算法的输出可如下表示：
 - $HMAC_k = H[(K^+ \oplus opad) \parallel H[(K^+ \oplus ipad) \parallel M]]$
- 算法的运行过程可描述如下：
 - ① K 的后面填充0以产生一个 b 比特长的 K^+ （例如 K 的长为160比特， $b=512$ ，则需填充 个零字节0x00）。

- 算法的输出可如下表示:

- $HMAC_k = H[(K^+ \oplus opad) || H[(K^+ \oplus ipad) || M]]$

- 算法的运行过程可描述如下:

- ① K 的左边填充0以产生一个 b 比特长的 K^+ (例如 K 的长为160比特, $b=512$, 则需填充44个零字节0x00)。
 - ② K^+ 与 $ipad$ 逐比特异或以产生 b 比特的分组 S_i 。
 - ③ 将 M 链接到 S_i 后。
 - ④ 将 H 作用于步骤③产生的数据流。
 - ⑤ K^+ 与 $opad$ 逐比特异或, 以产生 b 比特长的分组 S_0 。
 - ⑥ 将步骤④得到的哈希值链接在 S_0 后。
 - ⑦ 将 H 作用于步骤⑥产生的数据流并输出最终结果。



- 注意
 - K^+ 与 $ipad$ 逐比特异或以及 K^+ 与 $opad$ 逐比特异或的结果是将 K 中的一半比特取反，但两次取反的比特的位置不同
 - 而 S_i 和 S_0 通过哈希函数中压缩函数的处理，则相当于以伪随机方式从 K 产生两个密钥



- 在实现HMAC时，可预先求出下面两个量（见图，虚线以左为预计算）：

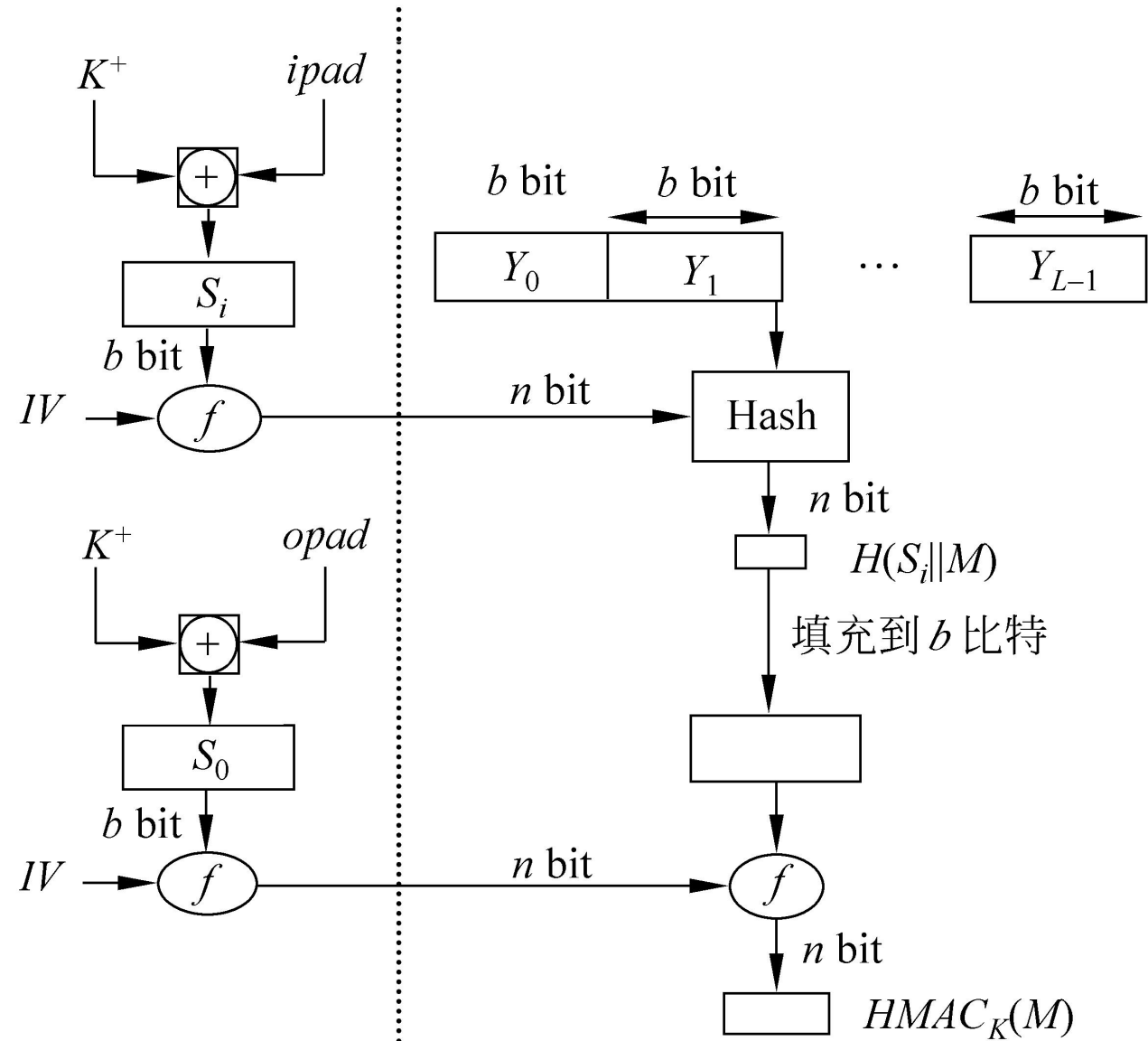
- $f(IV, (K^+ \oplus ipad))$

- $f(IV, (K^+ \oplus opad))$

- 其中 $f(cv, block)$ 是哈希函数中的压缩函数，

- 其输入是 n 比特的链接变量和 b 比特的分组
- 输出是 n 比特的链接变量。

- 这两个量的预先计算只在每次更改密钥时才需进行。事实上这两个预先计算的量用于作为哈希函数的初值IV。



HMAC的安全性

- 基于密码哈希函数构造的MAC的安全性取决于镶嵌的哈希函数的安全性
- HMAC最吸引人的地方是
 - 其设计者已证明了算法强度和嵌入的散列函数强度之间的确切关系
 - 即对HMAC的攻击等价于对内嵌哈希函数的下述两种攻击之一：
- ① 攻击者能够计算压缩函数的一个输出，即使IV是随机的和秘密的
 - 在第一种攻击中，可将压缩函数视为与哈希函数等价，而哈希函数的n比特长IV可视为HMAC的密钥。
 - 对这一散列函数的攻击可通过对密钥的穷搜索来进行
 - 攻击的复杂度为 $O(2^n)$



- ② 攻击者能够找出哈希函数的碰撞，即使IV是随机的和秘密的
 - 第二种攻击指攻击者寻找具有相同哈希值的两个消息，因此就是第II类生日攻击。
 - 对哈希值长度为n的哈希函数来说，攻击的复杂度为 $O(2^{n/2})$ 。
 - 因此第二种攻击对MD5的攻击复杂度为 $O(2^{64})$
 - 就现在的技术来说，这种攻击是可行的。但这是否意味着MD5不适合用于HMAC？



- 回答是否定的，原因如下：
 - 攻击者在攻击MD5时，可选择任何消息集合后离线寻找碰撞。由于攻击者知道哈希算法和默认的IV，因此能为自己产生的每个消息求出哈希值。
 - 然而，在攻击HMAC时，由于攻击者不知道密钥K，从而不能离线产生消息和认证码对
- 所以攻击者必须得到HMAC在同一密钥下产生的一系列消息，并对得到的消息序列进行攻击。
 - 对长128比特的哈希值来说，需要得到用同一密钥产生的 2^{64} 个分组（ 2^{73} 比特）。在1Gbit/s的链路上，需250000年，因此MD5完全适合于HMAC，而且就速度而言，MD5要快于SHA作为内嵌哈希函数的HMAC。



感谢聆听!

liaoyj@uestc.edu.cn
